UAM OIDC Relying Party Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAM OIDC Relying Party Implementation Guide..... 3
2. Introduction to UAM OIDC Relying Party Implementation.....4
3. OIDC Relying Party Servlet Implementation.....11
  3.1. Use OIDC Relying Party Servlet for Authentication.....28
  3.2. Use OIDC Relying Party Servlet for Reauthentication.....36
4. UAM OIDC Relying Party Configuration.....41
  4.1. Configurations for Authentication..... 43
  4.2. Configurations for Reauthentication..... 59
  4.3. Common Configurations.....70
5. OIDC Relying Party Implementation using API..... 82
6. UAM OIDC Relying Party Reports.....95
7. Appendices.....98

1. Preface - UAM OIDC Relying Party Implementation Guide

This document provides the concept, implementation steps and API integration details of AccessMatrix Universal Access Management (UAM) server as the OpenID Connect (OIDC) Relying Party (RP).

Intended Reader

This guide is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers.

Document Scope

This document provides the necessary steps to configure OIDC Endpoints and OAuth 2.0 Flows with details on its concept and API implementation.

> **Note:** The term *<ACMATRIX_ROOT>* will be used to refer to the root or installation directory of AccessMatrix on both Windows and Unix platforms.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *AccessMatrix REST API Specification*

2. Introduction to UAM OIDC Relying Party Implementation

OpenID Connect (OIDC) is a protocol for authenticating users, built on top of the OAuth 2.0 authorization framework. There are two primary actors in all OIDC interactions: the OpenID Provider (OP) and the Relying Party (RP).

- The OP is an OAuth 2.0 server that is capable of authenticating the end user and providing information about the result of the authentication and the end user to the RP.
- The RP is an OAuth 2.0 application that relies on the OP to handle authentication requests. It requires user authentication and claims from an OP.

AccessMatrix UAM can act as an OpenID Provider or as a Relying Party. In this implementation, the AccessMatrix UAM server acts as the OIDC Relying Party.

OIDC Relying Party Modules

AccessMatrix UAM provides OIDC Relying Party (RP) modules as connector/authentication modules for the following use cases:

- **Authentication**
  User authentication via the OIDC Authorization Code flow (optionally, with PKCE) with AccessMatrix UAM as the OIDC RP.
- **Reauthentication**
  User reauthentication via the OIDC Authorization Code flow (optionally, with PKCE) with AccessMatrix UAM as the OIDC RP. Currently, user reauthentication is supported for Single Sign-On for AccessMatrix Universal Sign-On (USO) applications only.

  > **Note:** The AccessMatrix Common Login Page (CLP) supports reauthentication for AccessMatrix Universal Sign-On (USO) applications. The full overview of how to configure external OP reauthentication with CLP for USO applications can be found at *AccessMatrix Common Login Page Deployment Guide - Configure Reauthentication via External OIDC OP*.

In each use case, the OIDC RP lookup module/OIDC RP login module prepares an authentication/reauthentication request to the OP respectively, and consumes the authorization code from the OP. The respective module then follows the OIDC standard to exchange the authorization code with an access token and ID token. The ID token is then used to receive the identity claim of the user as identified and authenticated by OP.

Next, AccessMatrix UAM looks up the user based on the identity claim to find the user data in AccessMatrix UAM. The lookup can be done using standard user information such as UUID and userId. It also uses AccessMatrix UAM Identity Mapping, a mechanism to map an external ID to an AccessMatrix user. This is done by storing the external ID as part of the identity mapping data of the user. This process of storing external ID is called "linking".

- For authentication, the OIDC Relying Party lookup module can be called via the Login API. This requires the application to have the OIDC Redirect URI endpoint to receive the authorization code from the OP (via browser), and then to call the Login API with the authorization code. To know more details about the API, refer to the API for Authentication section.
- For reauthentication, the OIDC Relying Party login module can be called via the Reauthenticate API. This requires the application to have the OIDC Redirect URI endpoint to receive the authorization code from the OP (via browser), and then to call the Reauthenticate API with the authorization code.

Another possible integration for both modules is to deploy the OIDC Relying Party Servlet, a separate UAM component, to send the authentication/reauthentication request to the OP via browser redirection, and also to act as the OIDC Redirect URI to receive the authorization code. This reduces the amount of integration in the application. However, this only fits for certain criteria. For more details, see the OIDC Relying Party Servlet section.

Authorization Code Flow vs. Authorization Code Flow with PKCE

The OIDC Relying Party (RP) modules support both the Authorization Code flow and the Authorization Code flow with Proof Key for Code Exchange (PKCE).

In the Authorization Code flow, the OAuth Client application (the RP) initiates a request for an authorization code from the authorization server (the OP), which is then exchanged for an access token from the OP. This flow is vulnerable to Cross-Site Request Forgery (CSRF) attacks if the OAuth Client is a browser-based application that uses an exposed client secret to facilitate the token exchange.

To overcome this vulnerability, you can make use of the AccessMatrix OIDC RP Servlet in your deployment. The OIDC RP Servlet manages the nonce and state mechanism within the flow to seamlessly prevent CSRF attacks.

Ideally, you should use the Authorization Code flow with PKCE, an extension of the Authorization Code flow designed to mitigate this attack. With the PKCE method, the vulnerability of using a client secret in the browser-based application is overcome by instead using PKCE elements at several steps of the flow. These elements include a cryptographically random key called the code verifier, and the code challenge, a transformed value derived from the code verifier. The code verifier is generated and securely cached in the AccessMatrix UAM back end to be used for the token exchange process.

A rogue application can only intercept the authorization code but is unable to access the code verifier, as it is secured within the AccessMatrix UAM back end and is not exposed to OAuth Client or even the OIDC RP Servlet. If the OIDC RP Servlet is not used, using the Authorization Code flow with PKCE method is necessary to mitigate CSRF attacks.

> **Note:** If the OIDC RP Servlet is used in your deployment, using the PKCE flow is strongly recommended but not mandatory. If the OIDC RP Servlet is not used, using the PKCE flow is mandatory.

Implementation Checklist

Below is the checklist for implementing the UAM OIDC Relying Party solution.

Authentication	**Tasks**	**Reference**
	Configure the Lookup Module	Configure Lookup Module
	Configure the Login Module	Configure Login Module
	Create the Authentication Realm	Configure Authentication Realm

Reauthentication	**Tasks**	**Reference**
	Configure the Login Module	Configure OIDC RP Login Module for User Reauthentication
	Assign the Login Module in USO Application Configuration	AccessMatrix Common Login Deployment Guide - Configure Reauthentication via External OIDC OP

Implementation Planning

Planning prior to environment setup is critical to the success of implementing the UAM OIDC Relying Party solution. Understanding the customer's nature of business, studying the customer's requirements and its existing environments will help to identify the actions to make prior to actual implementation.

The following sections discuss the implementation considerations that should be assessed during planning.

Security Consideration

Authentication	• **CSRF Prevention** To prevent Cross-Site Request Forgery (CSRF), the Relying Party needs to implement OAuth state properly. Normally the state is a randomly generated string that needs to be tied to the User Agent session. For UAM, the state is generated and stored in a cache on the Relying Party server by default. When the Relying Party receives the authorization code from OpenID Provider, the state will also be provided. The Relying Party needs to validate the state received from the OpenID Provider against the state stored earlier in the cache. The AccessMatrix OIDC Relying Party module helps to process the authorization code and validate what can be validated. • **Nonce** The OIDC Relying Party modules have the ability to generate a nonce, as well as validate the nonce inside the ID token. If the Relying Party application wants to generate its own nonce as part of browser/user agent session management, then it can also be supported. When calling Preauthenticate REST API, the Relying Party application can choose to provide its own nonce. In this case, the OIDC Relying Party module will not generate a new nonce. When the Relying Party application calls the login REST API to process the authorization code, it also needs to provide the expected nonce so that the module can validate accordingly. • **Encrypted JWT (JWE) for ID Token** The OIDC Relying Party modules support encrypted JWT for ID tokens. Thus, if the claims contain sensitive user information, this option can be used to protect a user's privacy or other information. However, this requires support from the OpenID Provider as well. • **Access Token, ID Token, and Refresh Token Management** By default, the OIDC RP lookup module does not store access tokens, ID tokens, or refresh tokens. It only uses ID tokens to resolve user identity in AccessMatrix. If the OIDC RP Servlet is not used, the Login API returns these tokens for the RP application to use (e.g. for API access to the Resource Server and for its own session management). Optionally, the OIDC RP lookup module can be configured to call the resource endpoint using the access token for user authorization. Alternatively, if the RP application is using the OIDC RP Servlet, it can also leverage the AccessMatrix back end for the session management of OIDC token(s), given that the RP application is using the AM5 session for its session management as well. To do so, when the Login API is called, the OIDC RP lookup module can be configured to persist the OIDC token(s). Likewise, when the AM5 Session Refresh API is called, if the persisted token(s) have expired, they can be refreshed as well.

	If the tokens are no longer required, the RP application can also invalidate the persisted OIDC token(s) by calling the Logout API. **Note:** While this feature is disabled by default, the OIDC RP lookup module can be configured to persist access and refresh tokens if required. Refer to Configure Session Management for more details.
Reauthentication	• **CSRF Prevention** To prevent Cross-Site Request Forgery (CSRF), the Relying Party needs to implement OAuth state properly. Normally the state is a randomly generated string that needs to be tied to the User Agent session. For UAM, the state is generated and stored in a cache on the Relying Party server by default. When the Relying Party receives the authorization code from OpenID Provider, the state will also be provided. The Relying Party needs to validate the state received from the OpenID Provider against the state stored earlier in the cache. The AccessMatrix OIDC Relying Party module helps to process the authorization code and validate what can be validated. • **Nonce** The OIDC Relying Party modules have the ability to generate a nonce, as well as validate the nonce inside the ID token. If the Relying Party application wants to generate its own nonce as part of browser/user agent session management, then it can also be supported. When calling Preauthenticate REST API, the Relying Party application can choose to provide its own nonce. In this case, the OIDC Relying Party module will not generate a new nonce. When the Relying Party application calls the login REST API to process the authorization code, it also needs to provide the expected nonce so that the module can validate accordingly. • **Encrypted JWT (JWE) for ID Token** The OIDC Relying Party modules support encrypted JWT for ID tokens. Thus, if the claims contain sensitive user information, this option can be used to protect a user's privacy or other information. However, this requires support from the OpenID Provider as well.

Integration Consideration

Authentication	• **Application has an existing integration with UAM and has been using WSASID cookie** This type of integration works well with OIDC RP Servlet. The servlet simplifies the integration and requires a minimum change at the application, as most of the OIDC process is done by the servlet and UAM. • **Forward Proxy** As part of OIDC process, UAM will need to make outgoing call to the OpenID Provider. In most deployments, UAM is deployed in a secure zone that may not have internet access. If the OpenID Provider is in the internet, UAM must be given access to make such a call to the OpenID Provider. Normally, this means a forward proxy is set up to allow HTTP Connect (HTTP tunneling) for UAM to setup TLS connection with the OpenID Provider. • **Use UAM to handle authorization code only (no session management and no Web SSO)**

	For this type of integration, it is best to call the UAM Preauthenticate and Login APIs directly. The Login API also returns the ID token claims along with the access token and refresh token. The OIDC `state` parameter must be handled and validated by the application front end to prevent CSRF. • **Use UAM to handle authorization code and to use UAM session management** If deploying OIDC RP Servlet is feasible, then using OIDC RP Servlet is an option to make integration easier. Alternatively, calling the UAM Preauthenticate and Login APIs is also a good option. The Login API returns the `sessionToken` for session management. • **Preauthenticate and Login API parameters** Both APIs allow flexible use of additional parameters, such as `nonce`, `expectedNonce`, `redirectURI`, and `scope`. These are optional parameters that allow the API caller to specify its own nonce or a different redirect URI. For simplicity, the OIDC Relying Party lookup module also allows configuration of some default values so that they do not need to be provided when calling the API - this simplifies the API call. For more details, refer to the *AccessMatrix REST API Specification*. • **Singapore SingPass Mobile Login** The OIDC Relying Party lookup module works well with Singapore SingPass Mobile Login. Other than the fact that it uses OIDC as the integration, OIDC Relying Party lookup module also supports the encrypted JWT (as ID token) and also it has a special configuration to extract the SingPass user's NRIC or UUID. For integration with SingPass Mobile, Singapore GovTech provides the ID token signing key as keystore. This needs to be first converted to JWK format as per the OIDC standard.
Reauthentication	• **Application has an existing integration with UAM and has been using WSASID cookie** This type of integration works well with OIDC RP Servlet. The servlet simplifies the integration and requires a minimum change at the application, as most of the OIDC process is done by the servlet and UAM. • **Forward Proxy** As part of OIDC process, UAM will need to make outgoing call to the OpenID Provider. In most deployments, UAM is deployed in a secure zone that may not have internet access. If the OpenID Provider is in the internet, UAM must be given access to make such a call to the OpenID Provider. Normally, this means a forward proxy is set up to allow HTTP Connect (HTTP tunneling) for UAM to setup TLS connection with the OpenID Provider. • **Use UAM to handle authorization code only (no session management and no Web SSO)** For this type of integration, it is best to call the UAM Preauthenticate and Reauthenticate APIs directly. The Reauthenticate API also returns the ID token claims, along with the access token and refresh token. The OIDC `state` parameter must be handled and validated by the application front end to prevent CSRF. • **Use UAM to handle authorization code and to use UAM session management** If deploying the OIDC RP Servlet is feasible, then using the OIDC RP Servlet is an option to make integration easier. Alternatively, calling the UAM

	Preauthenticate and Reauthenticate APIs is also a good option. The Login API returns the sessionToken for the session management. • **Preauthenticate API parameters** The Preauthenticate API allows the flexible use of additional parameters, such as nonce, expectedNonce, redirectURI, and scope. These are optional parameters that allow the API caller to specify its own nonce or a different redirect URI. For simplicity, the OIDC Relying Party login module also allows the configuration of some default values so that they do not need to be provided when calling the API - this simplifies the API call. For more details, refer to the *AccessMatrix REST API Specification*.

Performance Consideration

- The OIDC Relying Party modules require minimal CPU processing for the ID token validation (asymmetric key signature verification). However, if the ID token is encrypted (JWE), then it will use more substantial processing, as general asymmetric private key decryption requires more processing. Other than this, UAM will need to perform a lookup in the database to find the user based on the user UUID, user Id, or identity mapping. In general, the lookup should be fast as it is based on indexed database data.
- The OIDC Relying Party Servlet requires minimal processing as most of the OIDC processing is done by UAM.

3. OIDC Relying Party Servlet Implementation

OIDC Relying Party (RP) Servlet is suitable for application that has been using or wanting to use UAM Web SSO session (WSASID) for session management or for SSO purpose. OIDC RP Servlet handles the OIDC communication with the OpenID Provider to remove the need to implement OIDC directly inside the application. This simplifies the OIDC integration. OIDC RP Servlet uses the AccessMatrix OIDC Relying Party modules for the OIDC process. The servlet has the capability to do authentication request to the OpenID Provider and also to receive the authorization code. Once the process is done, OIDC RP Servlet sets the AccessMatrix session in WSASID cookie and redirect to the application. This also means that the application and the OIDC RP Servlet must share the same domain so that it can have access to the WSASID cookie. The application can use the WSASID cookie to make an API call to AccessMatrix Server to verify the user session and to get the user identity. Alternatively, AccessMatrix Web Service Agent Reverse Proxy (WSARP) can also be used to automatically verify the WSASID cookie.

OIDC RP Servlet only supports the Authorization Code flow and the Authorization Code flow with PKCE as these are the most suitable flows for browser-based web applications with server-side components.

Deployment Steps

1. Secure the OIDC RP WAR file from the AccessMatrix CD-Image. The filename format is "AccessMatrix-x.x.xxxx-GA-oauth-rp.war", e.g. **AccessMatrix-5.6.6.6609-GA-oauth-rp.war**.
2. (Optional) For easier reference, rename the WAR file to **oauth-rp.war**.
3. OIDC RP is a stateless web service, therefore you can deploy the WAR file (**oauth-rp.war**) to any web application container supported by AccessMatrix core products, e.g. Apache Tomcat.
4. If using Tomcat, create an *oauth-rp* folder under *<ACMATRIX_ROOT>**tomcat**webapps*\.
5. Extract the content of **oauth-rp.war** to this folder.

OIDC RP Components

Below is the list of the folders and sub-folders inside the OAuth RP WAR file:

Folder Name/File Name	Sub-folder/File Name	Description
assets	ico\login_failed.png	Contains the error icon.

Folder Name/File Name	Sub-folder/File Name	Description
css	style.css	CSS file for the default error page.
WEB-INF	classes\log4j2.properties	Contains the configurations of log level and log output file location.
	lib	The libraries used for OAuth RP.
	web.xml	Contains the configuration of REST connection to AccessMatrix server, cookies setting, and servlet mapping URL pattern for the RP servlets.
error.jsp		The page to display an error message.

Note: Only retain the "WEB-INF" folder and "error.jsp" when deploying into the production environment.

Configure OIDC RP Configuration File

To configure OIDC RP configuration file, navigate to *<ACMATRIX_ROOT>\tomcat\webapps\oauth-rp\WEB-INF* and modify the **web.xml** file. The content is shown below:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
    <context-param>
        <param-name>restAPIURL</param-name>
        <param-value>http://localhost:8080/am5/rest</param-value>
    </context-param>
    <context-param>
        <!-- This setting is only required if RP Servlet is
deployed with CLP -->
        <param-name>stateCookieTimeOutInSec</param-name>
        <param-value>180</param-value>
    </context-param>
        <context-param>
            <param-name>sessionTokenCookieName</param-name>
            <param-value>WSASID</param-value>  <!-- cookie name,
default value is WSASID -->
        </context-param>
    <context-param>
        <param-name>sessionTokenCookieMaxAgeInSec</param-name>
        <param-value>-1</param-value>  <!-- cookie will expire
```

```
after browser is closed if value is -1 -->
    </context-param>
    <context-param>
        <param-name>cookiePath</param-name>
        <param-value>/oauth-rp</param-value>
    </context-param>
    <context-param>
        <!--Allow cookie to set in this domain. If you using
Internet Explorer, this value is required to be set. Default is
empty-->
        <param-name>cookieDomain</param-name>
        <param-value></param-value>
    </context-param>
    <context-param>
        <param-name>cookieSameSite</param-name>
        <param-value></param-value> <!-- Default is not set for
SameSite -->
    </context-param>
    <context-param>
        <param-name>errorPageUrl</param-name>
        <param-value>/oauth-rp/error.jsp</param-value>
    </context-param>
    <!-- Connection Pool Configuration -->
<!--<context-param>
    <param-name>am.hc4.timeout.conn</param-name>
    <param-value>20000</param-value>
    </context-param>
    <context-param>
    <param-name>am.hc4.pool.maxconn_per_route</param-name>
    <param-value>50</param-value>
    </context-param>-->
    <servlet>
        <servlet-name>OAuthRPInitServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPInitServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPCallBackServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPCallBackServlet</servlet-class>
        <!-- Use "sessionTokenCookieDomain_<n>" to match cookie
domain with the corresponding realm Id(s).
        n is a number which begins with 1 and it can be
incremented if there are different sets of cookie domain and
realmId(s) mapping.
        If "sessionTokenCookieDomain_<n>" is set, it will
        override global settings of "cookieDomain" specified in
the context params -->
        <init-param>
            <param-name>sessionTokenCookieDomain_1</param-name>
            <param-value></param-value>
        </init-param>
        <!-- "realmIdsOfCookieDomain_<n>" is used to specify the
realmId(s)
        (multiple values is separated by e.g. "40485,40487") to be
applied with cookie domain
        specified under sessionTokenCookieDomain_1 param-name-->
```

```
        <init-param>
            <param-name>realmIdsOfCookieDomain_1</param-name>
            <param-value></param-value>
        </init-param>
        <!--<init-param>
            <param-name>sessionTokenCookieDomain_2</param-name>
            <param-value></param-value>
        </init-param>
         <init-param>
            <param-name>realmIdsOfCookieDomain_2</param-name>
            <param-value></param-value>
        </init-param> -->
    </servlet>
    <servlet>
        <servlet-name>OAuthRPJwksServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPJwksServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPReauthnInitServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPReauthnInitServlet</servlet-
class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPReauthnCallBackServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPReauthnCallBackServlet</servlet-
class>
    </servlet>
    <servlet>
        <servlet-name>OAuthRPReauthnJwksServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OAuthRPReauthnJwksServlet</servlet-
class>
    </servlet>
    <servlet>
        <servlet-name>OIDCRPCIBAPingCallBackServlet</servlet-name>
        <servlet-
class>com.isprint.oauth.rp.OIDCRPCIBAPingCallBackServlet</servlet-
class>
    </servlet>

    <servlet-mapping>
        <servlet-name>OAuthRPInitServlet</servlet-name>
        <!--Servlet path "/init" followed by "/<realmId>".
Configure the realmId where the lookup module is an OIDC Relying
Party Lookup Module handler -->
        <url-pattern>/rp/init/oidc</url-pattern>
        <!-- Servlet path to handle init REST API -->
        <url-pattern>/rp/rest/init</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPCallBackServlet</servlet-name>
        <!--Servlet path "/cb" followed by "/<realmId>". Configure
the realmId where the lookup module is an OIDC Relying Party
Lookup Module handler -->
```

```
        <url-pattern>/rp/cb/oidc</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPJwksServlet</servlet-name>
        <!--Servlet path "/<realmId>" followed by "/.well-
known/jwks.json". Configure the realmId where the lookup module is
an OIDC Relying Party Lookup Module handler -->
        <url-pattern>/oidc/.well-known/jwks.json</url-pattern>
        <!--Servlet path "/oidcrpmia/<moduleId>" followed by
"/.well-known/jwks.json". Configure the moduleId where the module
is an MyInfoApplication -->
        <url-pattern>/oidcrpmia/v4/.well-known/jwks.json</url-
pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPReauthnInitServlet</servlet-name>
        <url-pattern>/rp/rest/reauthn/init</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPReauthnCallBackServlet</servlet-name>
        <url-pattern>/rp/reauthn/cb/*</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OAuthRPReauthnJwksServlet</servlet-name>
        <!--Servlet path "/reauthn/<moduleId>" followed by
"/.well-known/jwks.json". Configure the moduleId where the login
module is an OIDC Relying Party Login Module handler -->
        <url-pattern>/reauthn/sgspstag/.well-known/jwks.json</url-
pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>OIDCRPCIBAPingCallBackServlet</servlet-name>
        <url-pattern>/rp/cb/ping</url-pattern>
    </servlet-mapping>

    <!-- Remove this remark to enable the filter
    <filter-mapping>
        <filter-name>SecureHeaderFilter</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

    <filter>
        <filter-name>SecureHeaderFilter</filter-name>
        <filter-
class>com.isprint.oauth.rp.SecureHeaderFilter</filter-class>
        <init-param>
            <param-name>ContentSecurityPolicy</param-name>
            <param-value>default-src 'self'</param-value>
        </init-param>
        <init-param>
            <param-name>ReferrerPolicy</param-name>
            <param-value>same-origin</param-value>
        </init-param>
        <init-param>
            <param-name>XPermittedCrossDomainPolicies</param-name>
            <param-value>none</param-value>
        </init-param>
```

```
    </filter>
    -->

    <!--
    <filter>
        <filter-name>httpHeaderSecurity</filter-name>
        <filter-
class>org.apache.catalina.filters.HttpHeaderSecurityFilter</filter
-class>
        <async-supported>true</async-supported>
        <init-param>
            <param-name>hstsEnabled</param-name>
            <param-value>true</param-value>
        </init-param>
        <init-param>
            <param-name>hstsMaxAgeSeconds</param-name>
            <param-value>31556927</param-value>
        </init-param>
        <init-param>
            <param-name>hstsIncludeSubDomains</param-name>
            <param-value>true</param-value>
        </init-param>
        <init-param>
            <param-name>antiClickJackingEnabled</param-name>
            <param-value>true</param-value>
        </init-param>
        <init-param>
            <param-name>antiClickJackingOption</param-name>
            <param-value>DENY</param-value>
        </init-param>
    </filter>
    <filter-mapping>
        <filter-name>httpHeaderSecurity</filter-name>
        <url-pattern>/*</url-pattern>
        <dispatcher>REQUEST</dispatcher>
    </filter-mapping>
    -->
</web-app>
```

The table below explains the important properties to configure in **web.xml** file:

Property Name	Description	Remarks
restAPIURL	This property refers to the AccessMatrix server REST API Service URL. Example: "http://localhost:8080/am5/rest"	This establishes the connection between OIDC RP web application and AccessMatrix server.
stateCookieTimeOutInSec	Specifies the cookie timeout for state in seconds.	

Property Name	Description	Remarks
	Note: This setting is only required if CLP is used.	
sessionTokenCookieName	Specifies the session token cookie name.	
sessionTokenCookieMaxAgeInSec	Specifies the cookie timeout for the session token in seconds. Ideally, the session token cookie should have a short timeout value. If the value specified is "-1", the cookie will expire after the browser is closed.	
cookiePath	Cookie path for session token and state.	Default: /oauth-rp
cookieDomain	Specifies the cookie domain for session token and state.	
cookieSameSite	Specifies the cookie domain, path, and secure flag for WSAWID cookie. This setting is specific to Session only for WSASID cookie. **Note:** SameSite value is case-sensitive.	Possible values: • "None" • "Strict" • "Lax"
errorPageUrl	Default error page for Relying Party Servlet endpoints (i.e. "rp/init/<realmId> or rp/cb/<realmId>") to display the error to end user. Example: http://localhost:8080/oauth-rp/error.jsp	
OAuthRPInitServlet url-pattern	Refers to the init servlet path. Parameter value should be: "/rp/init/<realmId>".	<realmId> refers to the lookup module using "OIDC Relying Party Lookup Module" implementation.
OAuthRPCallBackServlet url-pattern	Refers to the callback servlet path. Parameter value should be: "/rp/cb/<realmId>".	<realmId> refers to the Id of the authentication realm whose lookup module is using the "OIDC Relying Party Lookup Module" implementation.

Property Name	Description	Remarks
OAuthRPJwksServlet url-pattern	Refers to the jwks servlet path. Parameter value should be: "/<realmId>/.well-known/jwks.json"	<realmId> refers to the lookup module using "OIDC Relying Party Lookup Module" implementation.
OAuthRPReauthnInitServlet url-pattern	Refers to the init servlet path for external OP reauthentication.	
OAuthRPReauthnCallBackServlet url-pattern	Refers to the callback servlet path for external OP reauthentication.	
OAuthRPReauthnJwksServlet url-pattern	Refers to the jwks servlet path. Parameter value should be: "/reauthn/<loginModuleId>/.well-known/jwks.json"	<loginModuleId> refers to the Id of the login module using the "OIDC Relying Party Login Module" implementation.

Save the changes.

Enable Client-authenticated SSL Support

To support client-authenticated SSL (a mandatory configuration for Production environment), the following settings should be configured:

- Modify the restAPIURL value to use secure connection to the AccessMatrix server as shown below:

```
XML

<context-param>
     <param-name>restAPIURL</param-name>
     <param-
value>https://localhost:8443/am5/rest</param-value>
</context-param>
```

- Configure the following client keystore properties:

```
XML

<context-param>
     <param-name>am.hc4.ssl.keystore</param-name>
     <param-value>/opt/acmatrix/tomcat/conf/clp-
client.keystore</param-value>
</context-param>
```

```
<context-param>
     <param-name>am.hc4.ssl.keystore-passphrase</param-name>
     <param-
value>AesEncryptor:pWtZZ0N6bfIvSlBBF0zXC0ZwS1Kny/GexGVFpaqWI1Q=</pa
ram-value>
</context-param>
```

Note: Refer to the *AccessMatrix Common Login Page Deployment Guide - List of Configurable Parameters* section for the description of the properties.

Update Session Token Cookie Name

The OIDC RP Servlet's session token cookie name can be customized as needed. This can help to avoid potential conflicts arising from various AccessMatrix components sharing the same session token cookie name.

1. Navigate to *<AM_ROOT>\tomcat\webapps\oauth-rp\WEB-INF* and open the **web.xml** file.

   Note: The actual location of the *oauth-rp* folder may vary depending on where you chose to deploy it. Refer back to the deployment instructions detailed in Deployment Steps if needed.

2. Search for the sessionTokenCookieName param-name:

   XML

   ```
   <context-param>
     <param-name>sessionTokenCookieName</param-name>
     <param-value>WSASID</param-value><!-- cookie name,
   default value is WSASID -->
   </context-param>
   ```

3. Replace the default value with the same name specified in the AccessMatrix **wpe_config.properties** file (e.g. "AM5RPCookie").

4. Save the changes.

OIDC RP Servlet Tomcat Hardening and Security Configurations

The following sections provide information on the additional OIDC RP Servlet configurations that are made for Tomcat hardening and security.

Enable HTTP Header Security Filter

The **HTTP Header Security** filter can be enabled to allow the addition of the following HTTP response headers:

- **Strict-Transport-Security**
- **X-Content-Type-Options**
- **X-Frame-Options**
- **X-XSS-Protection**

To enable this filter, search for the httpHeaderSecurity element in the OIDC RP **web.xml** file and uncomment the remark as follows:

XML

```
  <filter>
    <filter-name>httpHeaderSecurity</filter-name>
    <filter-
class>org.apache.catalina.filters.HttpHeaderSecurityFilter</filter
-class>
    <async-supported>true</async-supported>
    <init-param>
      <param-name>hstsEnabled</param-name>
      <param-value>true</param-value>
    </init-param>
    <init-param>
      <param-name>hstsMaxAgeSeconds</param-name>
      <param-value>31556927</param-value>
    </init-param>
    <init-param>
      <param-name>hstsIncludeSubDomains</param-name>
      <param-value>true</param-value>
    </init-param>
    <init-param>
      <param-name>antiClickJackingEnabled</param-name>
      <param-value>true</param-value>
    </init-param>
    <init-param>
      <param-name>antiClickJackingOption</param-name>
      <param-value>DENY</param-value>
    </init-param>
  </filter>
  <filter-mapping>
    <filter-name>httpHeaderSecurity</filter-name>
    <url-pattern>/*</url-pattern>
    <dispatcher>REQUEST</dispatcher>
  </filter-mapping>
```

Once enabled, the following response headers and default values will be set:

Header	Default Value	Description

Header	Default Value	Description
Strict-Transport-Security		In this case, the default value is 31556927 seconds (over 1 year).
	includeSubDomains	Applies HSTS to all subdomains as well.
X-Content-Type-Options	nosniff	Disables MIME sniffing, meaning the browser uses the MIME type declared by the server when rendering a file.
X-Frame-Options	DENY	Prevents pages from being displayed in frames. All of the browser's attempts to load a page in a frame will fail, whether the page is from the same site or other sites.
X-XSS-Protection	1; mode=block	Enables XSS filtering. The browser will not render the page if an XSS attack is detected.

See the following sub-sections for information on each response header.

Notes:
- It is recommended that you do not modify the default values provided.
- Detailed explanations of the individual response headers and all of their configurable values are beyond the scope of this guide.

Strict-Transport-Security

The **Strict-Transport-Security** response header informs the browser that the site should only be accessed using HTTPS. Future attempts to access the site using HTTP will automatically be converted to HTTPS as well. This is also referred to as the **HTTP Strict Transport Security (HSTS)** header.

By default, the header is set with the "max-age=31556927" and "includeSubDomains" values. This applies HSTS to the site's parent domain and all its subdomains for a duration of 31556927 seconds.

The response header's values can be modified by configuring the values of the hstsMaxAgeSeconds and hstsIncludeSubDomains parameters in the **web.xml** file.

X-Content-Type-Options

The **X-Content-Type-Options** response header determines whether or not the browser "sniffs" files to determine if their actual MIME type (file type) is different from what is declared by the server (e.g. an HTML file presented as a JPG file), so as to render them using the actual file type.

By default, the header is set with the "nosniff" value. This disables MIME sniffing and forces the browser to use the file type declared by the server when rendering the file, regardless of the actual file type. It is used to secure the user against MIME sniffing vulnerabilities, where the browser renders malicious files disguised as different MIME types using their actual MIME types.

To enable MIME sniffing, insert the additional blockContentTypeSniffingEnabled parameter with the "false" value into the **web.xml** file as follows:

XML

```
  <filter>
    <filter-name>httpHeaderSecurity</filter-name>
    <filter-
class>org.apache.catalina.filters.HttpHeaderSecurityFilter</filter
-class>
    <async-supported>true</async-supported>
    <init-param>
      <param-name>hstsEnabled</param-name>
      <param-value>true</param-value>
    </init-param>
...
    <init-param>
      <param-name>blockContentTypeSniffingEnabled</param-name>
      <param-value>false</param-value>
    </init-param>
  </filter>
```

X-Frame-Options

The **X-Frame-Options** response header determines if the browser can render a page in a frame.

By default, the header is set with the "DENY" value. This restricts the browser from rendering pages in frames, which can protect against clickjacking attacks where users are tricked into clicking on invisible or disguised webpage elements with malicious links.

The response header value can be modified by configuring the value of the antiClickJackingOption parameter. The response header can also be disabled by setting the value of the antiClickJackingEnabled parameter to "false".

X-XSS-Protection

The **X-XSS-Protection** response header determines if cross-site scripting (XSS) filtering is enabled.

By default, the header is set with the "1; mode=block" value. This prevents the rendering of a page if an XSS attack - where malicious client-side code is injected into a website - is detected.

To disable XSS filtering, insert the additional xssProtectionEnabled parameter with the "false" value into the **web.xml** file as follows:

XML

```
  <filter>
    <filter-name>httpHeaderSecurity</filter-name>
    <filter-
class>org.apache.catalina.filters.HttpHeaderSecurityFilter</filter
-class>
    <async-supported>true</async-supported>
    <init-param>
      <param-name>hstsEnabled</param-name>
      <param-value>true</param-value>
    </init-param>
...
    <init-param>
      <param-name>xssProtectionEnabled</param-name>
      <param-value>false</param-value>
    </init-param>
  </filter>
```

Enable Secure Header Filter

The **Secure Header Filter** can be enabled to allow the addition of the following HTTP response headers:

- **Content-Security-Policy**
- **X-Permitted-Cross-Domain-Policies**
- **Referrer-Policy**

To do so, search for the SecureHeaderFilter element in the **web.xml** file and uncomment the remark as follows:

XML

```
  <filter-mapping>
    <filter-name>SecureHeaderFilter</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

  <filter>
    <filter-name>SecureHeaderFilter</filter-name>
    <filter-
```

```
class>com.isprint.oauth.rp.SecureHeaderFilter</filter-
class>
    <init-param>
      <param-name>ContentSecurityPolicy</param-name>
      <param-value>default-src 'self'</param-value>
    </init-param>
    <init-param>
        <param-name>ReferrerPolicy</param-name>
        <param-value>same-origin</param-value>
    </init-param>
    <init-param>
      <param-name>XPermittedCrossDomainPolicies</param-
name>
      <param-value>none</param-value>
    </init-param>
  </filter>
```

Once enabled, the following response headers and default values will be set:

Header	Default Value	Description
Content-Security-Policy	default-src ‘self’	Disallows certain resources from being loaded from the page. In this case, only resources from the site's origin are allowed.
X-Permitted-Cross-Domain-Policies	none	Restricts all cross-domain policy files on the target server. This blocks all cross-domain requests.
Referrer-Policy	same-origin	Allows the sending of referrer information (the origin, path, and query string) for same-origin requests only.

See the following sub-sections for information on each response header.

Notes:
- It is recommended that you do not modify the default values provided.
- Detailed explanations of the individual response headers and all of their configurable values are beyond the scope of this guide.

Content-Security-Policy

The **Content-Security-Policy** response header tells the browser which content sources it is allowed to load resources from.

By default, the header is set with the "default-src ‘self’" value. This restricts the resources that are allowed to be loaded to those from the site's origin only. It is used to protect against XSS attacks.

The response header value can be modified by configuring the value of the ContentSecurityPolicy parameter in the **web.xml** file.

X-Permitted-Cross-Domain-Policies

The **X-Permitted-Cross-Domain-Policies** response header determines which cross-domain policy files are allowed. Cross domain policy files provide permissions for cross-domain requests from requested content (namely, Adobe Flash applications and PDF documents).

By default, the header is set with the "none" value. This means that no cross-domain policy files are allowed. It is used to prevent others from embedding data from your site into Adobe Flash applications or PDF documents, etc.

The response header value can be modified by configuring the value of the XPermittedCrossDomainPolicies parameter in the **web.xml** file.

Referrer-Policy

The **Referrer-Policy** response header determines how much referrer information (information about the page the user originated from before arriving at the target site) is included with HTTP requests.

By default, the header is set with the "same-origin" value. This enables the inclusion of referrer information (the origin, path, and query string) for requests made from the same origin only.

The response header value can be modified by configuring the value of the ReferrerPolicy parameter in the **web.xml** file.

Enable Other Security Features

Client-initiated Renegotiation

Secure Sockets Layer (SSL)/Transport Layer Security (TLS) client-initiated renegotiation allows the client to renegotiate different encryption parameters for an SSL/TLS connection within the same TCP connection. This leaves the server vulnerable to a Denial of Service (DoS) attack, where multiple renegotiation processes are triggered in a single TCP connection to overwhelm the server and render the service unavailable.

By default, client-initiated renegotiation is not disabled. To disable it, locate the **setenv.bat** file in *<ACMATRIX_ROOT>\tomcat\bin* and insert the -Djdk.tls.rejectClientInitiatedRenegotiation=true Java property as follows:

Java

```
:setJavaOpts
set TITLE=AccessMatrix Server
set
"LOCALLIBPATH=%JAVA_HOME_LIBPATH%;bin;%PATH%;%LOCALLIBPATH%"

rem AM-10739: moved GC args to its own section below
set JAVA_OPTS=%JAVA_OPTS% -Xms400m -Xmx1g "-
Djava.library.path=%LOCALLIBPATH%"
 -Dderby.system.home=db -Dam.ehcache.set_rmi_ip_addr=true -
XX:MaxMetaspaceSize=256M
 -XX:NewSize=128M -XX:+HeapDumpOnOutOfMemoryError -
XX:HeapDumpPath=logs
 -Dhost.name=%COMPUTERNAME% -
Djavax.net.ssl.trustStore=conf/amtrust.keystore
 -Djavax.net.ssl.trustStorePassword=passphrase -
Djdk.tls.rejectClientInitiatedRenegotiation=true
```

Note: This feature is only applicable for JDK 13 and onward.

Session Resumption

TLS session resumption enables the sharing of TLS session information between multiple connections. This provides greater connection speeds, as the client resuming their TLS session will not need to undergo the full TLS handshake process again. If enabled, it is possible for attackers to steal existing TLS sessions from other users.

By default, session resumption is not disabled. To disable it, locate the setenv.bat file in *<ACMATRIX_ROOT>\tomcat\bin* and insert the -Djdk.tls.client.enableSessionTicketExtension=false and -Djdk.tls.server.enableSessionTicketExtension=false Java properties as follows:

Java

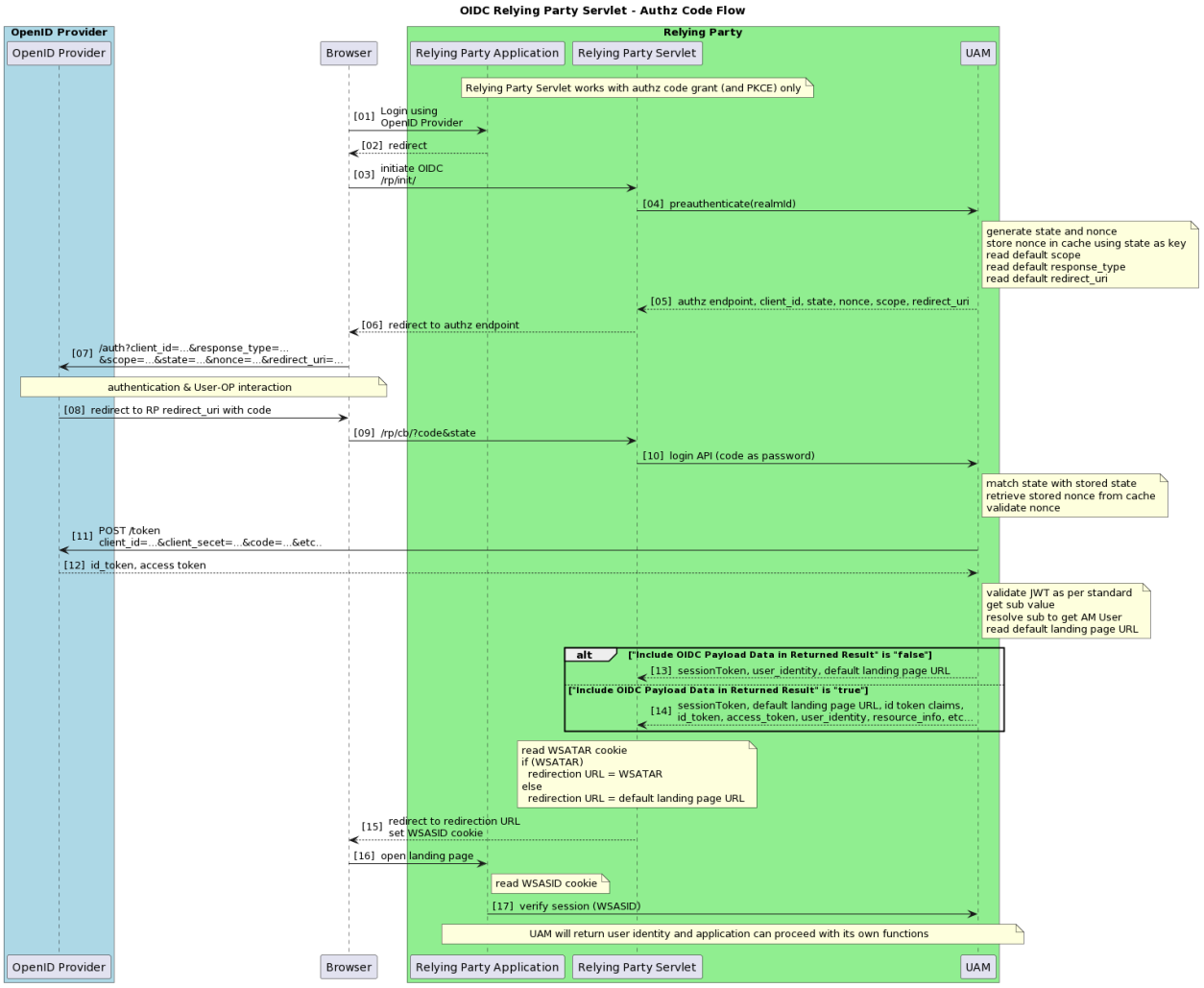
```
:setJavaOpts
set TITLE=AccessMatrix Server
set
"LOCALLIBPATH=%JAVA_HOME_LIBPATH%;bin;%PATH%;%LOCALLIBPATH%"

rem AM-10739: moved GC args to its own section below
set JAVA_OPTS=%JAVA_OPTS% -Xms400m -Xmx1g "-
Djava.library.path=%LOCALLIBPATH%"
 -Dderby.system.home=db -Dam.ehcache.set_rmi_ip_addr=true -
```

```
XX:MaxMetaspaceSize=256M -XX:NewSize=128M
 -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=logs -
Dhost.name=%COMPUTERNAME%
 -Djavax.net.ssl.trustStore=conf/amtrust.keystore -
Djavax.net.ssl.trustStorePassword=passphrase
 -Djdk.tls.client.enableSessionTicketExtension=false -
Djdk.tls.server.enableSessionTicketExtension=false
```

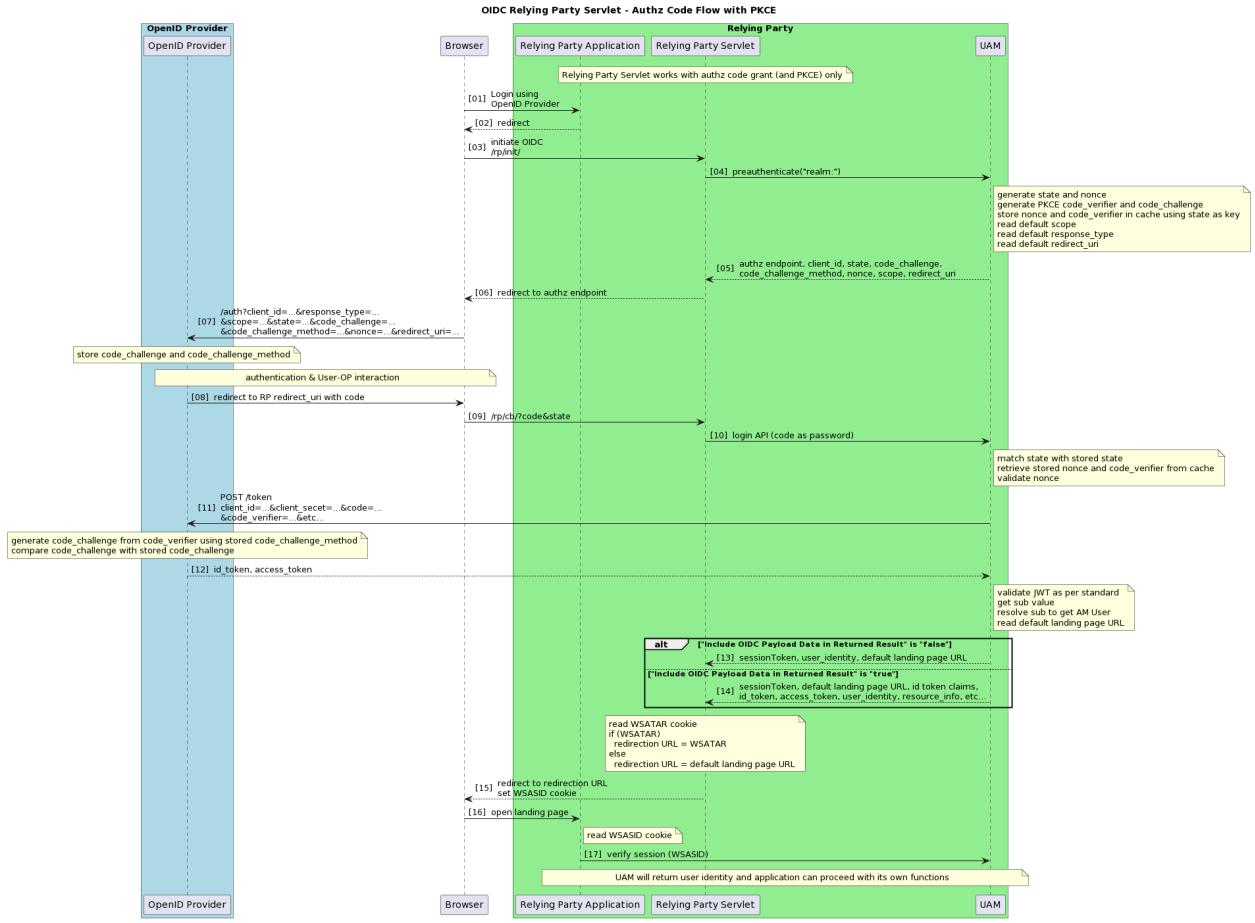
Note: This feature is only applicable for JDK 14 and onward.

3.1. Use OIDC Relying Party Servlet for Authentication

OIDC Relying Party Servlet with Authorization Code Flow Sequence Diagram



OIDC Relying Party Servlet with Authorization Code Flow with PKCE Sequence Diagram



OIDC Relying Party Endpoints

There following are the endpoints for Relying Party Servlets used for authentication:

- **OIDC initiation (/rp/init/<realmId>):**
    - This endpoint is used to trigger OIDC authentication request process. It makes a call to AccessMatrix's preauthenticate API to generate the state and nonce, and to get information on the OpenID Provider as configured in the lookup module.
    - If the OIDC RP Servlet is deployed together with CLP, the state obtained from the Preauthenticate API will be set as a cookie and stored on the user's browser instead.
- **OIDC redirect URI/callback URI (/rp/cb/<realmId>):**
    - This endpoint is called by the OpenID Provider via the browser to provide the authorization code. Upon receiving the authorization code, the endpoints validate the

state for CSRF prevention. The authorization code is also sent to AccessMatrix to get the ID token from the OpenID Provider for the Single Sign-On process. If the PKCE flow is enabled, the code verifier is sent along with the authorization code in exchange for the ID token. After receiving AccessMatrix session token from AccessMatrix Server, the endpoint will send a redirection request to the browser to open the Relying Party application landing page. The WSASID cookie containing the AccessMatrix session token is also set into the browser so that Relying Party server-side application can use it as the authentication session and validate it by calling the AccessMatrix REST API.

- **OIDC JSON Web Keys (JWKS) (<realmId>/.well-known/jwks.json):**
    - This endpoint is used to retrieve the Relying Party encryption public keys. OpenID Provider may call this endpoint to get the encryption keys used to encrypt the ID token.

The supported OIDC Relying Party lookup module Id and realm Id must be configured in the **web.xml** under the corresponding servlet mappings. Refer to Configure OIDC RP Configuration File for information on how to locate and configure the settings.

Initiation/Authentication Request Trigger URI - rp/init/<realmId>

HTTP Method: GET/POST

Path: ${yourOAuthRPServlet}/rp/init/<realmId>

This endpoint prepares and redirects the Relying Party application for OIDC authentication. This includes the generation of nonce and state, as well as the PKCE code challenge and code challenge method if applicable. This also includes the preparation of the request parameters and the redirection of the user to the OpenID Provider for authentication.

Request Parameters

Parameter	Data Type	Description	Mandatory
scope	String	Value is "openid". It indicates that the token endpoint (/token) should return an ID token.	FALSE

Examples

Request Example

Request Example
Code
`https://yourOAuthRPServlet/rp/init/<realmId>`

Redirect URI - rp/cb/<realmId>

HTTP Method: GET/POST

Path: ${yourOAuthRPServlet}/rp/cb/<realmId>

This endpoint is a callback URL to be used by the OpenID Provider to return the user's authentication response after the user has been authenticated or authorized. Before the exchange of the OIDC token(s) is processed, the state provided by the OpenID Provider is first matched with the state cached on the Relying Party server to prevent CSRF attacks. Once the state has been successfully validated, the endpoint will process the exchange of the authorization code with the OIDC token(s). If the PKCE method is used, the code verifier is passed in with the authorization code. Subsequently, the endpoint will look up the user in the AccessMatrix server and map it with the user identity obtained from the ID token. Upon successful authentication, the user will be redirected to a default landing page or the target URL of the Relying Party application.

Request Parameters

If the OIDC flow is authenticated successfully, then OpenID Provider will call this endpoint with the code and state parameters.

Parameter	Data Type	Description	Mandatory
code	String	The authorization code provided by the OpenID Provider	TRUE
state	String	The state generated earlier by rp/init/<realmId> and then returned back by the OpenID Provider for CSRF mitigation	TRUE

When an error occurs during the processing of the authorization code, this endpoint will redirect to an error page (as configured in the OIDC Relying Party lookup module's **Error Page URL** attribute) with the error information as request parameters. The error page can display the error accordingly.

Parameter	Data Type	Description
error	String	The error constant indicating the type of error
error_description	String	The short description about the error
state	String	The state generated earlier by rp/init/<realmId> and then returned back by the OpenID Provider for CSRF mitigation

The possible errors are listed below.

error	Explanation
invalid_user	User cannot be found/resolved in the system based on the configured identity mapping. Some possible messages that can be displayed in the error page: • Your account is not registered in the system, you can register here. • You do not have access to the <application name>, contact the administrator. • (SingPass Mobile) Your SingPass account is not registered in this application, contact the application administrator.
expired_login	Login session has expired. It is caused by expired/invalidated nonce or state. If PKCE flow is used, it is caused by expired/invalidated state. It is better to ask the user to redo the login process. Some possible messages that can be displayed in the error page: • The login session has expired. You must login again.
system_error	An error that cannot be resolved by the user. A configuration problem most likely causes this. Another possibility is that a problem occurred when contacting the OpenID Provider. More details are logged in the OIDC Relying Party Servlet and AccessMatrix log files.
op_system_error	A server error that is thrown from the OpenID Provider. Most likely, the authorization server encountered an unexpected condition that prevented it from fulfilling the request. Below is the message that is displayed in the error page: • The authentication provider (OpenID Provider) encounters a server error at the moment.

Examples

Request Example

Code

```
https://yourOAuthRPServlet/rp/cb/<realmId>?code=...&state=...
```

JSON Web Keys (JWKS) URI - <realmId>/.well-known/jwks.json

HTTP Method: GET

Path: ${yourOAuthRPServlet}/<realmId>/.well-known/jwks.json

This endpoint provides the Relying Party's public keys that are used for encryption. This endpoint can be called by OpenID Provider to automatically retrieve the encryption keys for encrypting ID token.

Examples

Request Example

Code

```
https://yourOAuthRPServlet/<realmId>/.well-known/jwks.json
```

Response Example (Success)

JSON


```
{
  "keys": [
    {"kty":"EC",
     "use":"enc",
     "crv":"P-256",
     "kid":"ORE-az_modid-D6X8rfBrYGg",
     "x":"Ly3oIFBpfJnaNIk8ab03vqrnRg78CJfkaRQF2NaMvKo",
     "y":"E3cmVLi-x1l7btlATO19kCX9oXnG_MMS2vgFvUEop4U",
     "alg":"ECDH-ES+A256KW"
    },
    {
      "kty":"EC",
      "use":"sig",
      "crv":"P-256",
      "kid":"ORS-az_modid-MyXILDhSJDk",
      "x":"VsOU8I7LZzAr5bU2ID0vewyYgx7IyZUGxKHZnX5vcJ0",
      "y":"oB8G3tElcr-YfhC0ZeiyF9_5KspLfCkvN01ZTUqOiiE",
      "alg":"ES256"
```

Request Example


```
    }
  ],
  "version":18
}
```


OIDC Relying Party API

This section describes the APIs to use for OIDC Relying Party implementation.

rp/rest/init

HTTP Method: GET/POST

Path: ${yourOAuthRPServlet}/rp/rest/init

This API is used by Relying Party to prepare its client application, prior to calling Authorization Endpoint for OIDC Authentication.

Response Parameters

Parameter	Description
state	The state value generated and cached in the backend of the UAM server. It is used to prevent CSRF attacks.
nonce	The nonce value generated and cached in the backend of the UAM server. This value is returned in the ID token and validated with cached value to mitigate replay attacks.
nonceStateExpiresAt	Nonce and state expiry time. Relying Party application should check this expiry time before calling authorization endpoint. If they have already expired, the application should call this endpoint to reinitialize the environment prior to calling OIDC Authorization Endpoint.
redirect_uri	The callback location where the user-agent will be redirected to along with the code.
client_id	The "OAuth Client Id" of the created OAuth Client in Admin Console.
code_challenge	The BASE64-URL-encoded string of the SHA256 hash of the code verifier. This is returned if and only if the Authorization Code flow with PKCE is used.

Parameter	Description
code_challenge_method	The method used to generate the code_challenge (e.g. S256). This is returned if and only if the Authorization Code flow with PKCE is used.
authzEndpointPrepared	The prepared external OP authorization endpoint URL for further authentication processes.

Sample Payload

Request Example

JSON


```
{
  "realmId":"oidc",
}
```


Response Example (Success)

JSON


```
{
    "nonceStateExpiresAt":"1578965335069",
    "state":"EyPCLnY-mpXcvsdG",
    "redirect_uri":"http://192.168.50.53:8080/oauth-rp/rp/cb/oidc",
    "nonce":"0-ZW1vviNarEHc1_",
    "client_id":"ZU1iWicoF66HJ3nO2gXbng",

"code_challenge":"HtKdpT%2B9ebYA%2Fai5uPugtKv%2FIX%2FUyn1y5cUDnWsyEiQ%3D",
    "code_challenge_method":"S256",
    "authzEndpointPrepared":"https://login.microsoftonline.com/1fdcc734-
b8b9-4090-ae50-
6e6bc3283c35/oauth2/v2.0/authorize?client_id=ZU1iWicoF66HJ3nO2gXbng&response
_type=code&scope=openid&redirect_uri=https%3A%2F%2Fi-
sprint.com%2Fclp%2Frp%2Fcb%2FoidcOP&nonce=CaVLk7t8UCV4gyU_&state=1RF2BRF7H3"
}
```

3.2. Use OIDC Relying Party Servlet for Reauthentication

OIDC Relying Party Endpoints

There following are the endpoints for Relying Party Servlets used for reauthentication:

- **OIDC initiation (/rp/rest/reauthn/init/<loginModuleId>):**
    - This endpoint is called via **HTTP POST** with a session token to trigger the OIDC reauthentication request process. It makes a call to AccessMatrix's Preauthenticate API to generate the state and nonce, and to get information on the OpenID Provider as configured in the login module. The session token and state received in the previous steps are stored as cookies for later use.
- **OIDC redirect URI/callback URI (/rp/reauthn/cb/<loginModuleId>):**
    - This endpoint is called by the OpenID Provider via browser redirection to provide the authorization code. This endpoint of OIDC Relying Party makes a call to AccessMatrix's Reauthenticate API to trigger the corresponding login module (configured in this section) to exchange the authorization code for OIDC token(s). If the PKCE flow is enabled, the code verifier is sent along with the authorization code in exchange for the ID token. A new session token and the Relying Party's default landing page URL (e.g. CLP's SSO landing page) are returned as the API response to complete the reauthentication process with the caller (e.g. CLP and USO portal).

The supported OIDC Relying Party login module ID must be configured in the OIDC RP **web.xml** file under the corresponding servlet mappings. Refer to Configure OIDC RP Configuration File for information on how to locate and configure the settings.

Reauthentication Request Trigger URI - rp/rest/reauthn/init

HTTP Method: POST

Path: ${yourOAuthRPServlet}/rp/rest/reauthn/init

This endpoint prepares and redirects the Relying Party application for OIDC reauthentication. This includes the generation of nonce and state, as well as the PKCE code challenge and code challenge method if applicable. This also includes the preparation of the request parameters and the redirection of the user to the OpenID Provider for reauthentication.

Request Parameter

Parameter	Data Type	Description	Mandatory
loginModuleId	String	The Id of the OIDC RP login module created and configured for the external OIDC reauthentication process.	TRUE
sessionToken	String	The session token used for the external OIDC reauthentication process.	TRUE

Examples

Request Example
Code
`https://yourOAuthRPServlet/rp/rest/reauthn/init`

Redirect URI - rp/reauthn/cb/<loginModuleId>

HTTP Method: GET

Path: ${yourOAuthRPServlet}/rp/reauthn/cb/<loginModuleId>

This endpoint is a callback URL to be used by the OpenID Provider to return the user's authentication response after the user has been reauthenticated. Before the exchange of the OIDC token(s) is processed, the state provided by the OpenID Provider is first matched with the state cached on the Relying Party server to prevent CSRF attacks. Once the state has been successfully validated, the endpoint will process the exchange of the authorization code for OIDC token(s). Subsequently, the endpoint will look up the user in the AccessMatrix server and map it with the user identity obtained from the ID token. Upon successful authentication, the user will be redirected to the Relying Party's default landing page URL (e.g. CLP's SSO landing page) .

Request Parameters

If the OIDC flow is authenticated successfully, then OpenID Provider will call this endpoint with the code and state parameters.

Parameter	Data Type	Description	Mandatory
code	String	The authorization code provided by the OpenID Provider	TRUE
state	String	The state generated earlier by rp/rest/reauthn/init/<loginModuleId> and then returned back by the OpenID Provider for CSRF mitigation	TRUE

When an error occurs during the processing of the authorization code, this endpoint will redirect the user to an error page (as configured in the OIDC Relying Party login module's **Error Page URL** attribute) with the error information as request parameters. The error page can display the error accordingly.

Parameter	Data Type	Description
error	String	The error constant indicating the type of error
error_description	String	The short description about the error
state	String	The state generated earlier by rp/rest/reauthn/init/<loginModuleId> and then returned back by the OpenID Provider for CSRF mitigation.

The possible errors are listed below.

error	Explanation
invalid_user	User cannot be found/resolved in the system based on the configured identity mapping. Some possible messages that can be displayed in the error page: • Your account is not registered in the system, you can register here. • You do not have access to the <application name>, contact the administrator. • (SingPass Mobile) Your SingPass account is not registered in this application, contact the application administrator.
expired_login	Login session has expired. It is caused by expired/invalidated nonce or state. If PKCE flow is used, it is caused by expired/invalidated state. It is better to

error	Explanation
	ask the user to redo the login process. Some possible messages that can be displayed in the error page: • The login session has expired. You must login again.
system_error	An error that cannot be resolved by the user. A configuration problem most likely causes this. Another possibility is that a problem occurred when contacting the OpenID Provider. More details are logged in the OIDC Relying Party Servlet and AccessMatrix log files.
op_system_error	A server error that is thrown from the OpenID Provider. Most likely, the authorization server encountered an unexpected condition that prevented it from fulfilling the request. Below is the message that is displayed in the error page: • The authentication provider (OpenID Provider) encounters a server error at the moment.

Examples

Request Example (Success)
Code
`https://yourOAuthRPServlet/rp/reauthn/cb/<loginModuleId>?code=...&state=...`

JSON Web Keys (JWKS) URI - reauthn/<loginModuleId>/.well-known/jwks.json

HTTP Method: GET

Path: ${yourOAuthRPServlet}/reauthn/<loginModuleId>/.well-known/jwks.json

This endpoint provides the Relying Party's public keys that are used for encryption. This endpoint can be called by OpenID Provider to automatically retrieve the encryption keys for encrypting ID token.

Examples

Request Example (Success)
Code

Request Example (Success)

```
https://yourOAuthRPServlet/reauthn/<loginModuleId>/.well-
known/jwks.json
```

4. UAM OIDC Relying Party Configuration

This section defines the steps relating to the implementation of AccessMatrix UAM as the OIDC Relying Party. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of UAM OIDC Relying Party solution. Then, obtain the necessary information from the OpenID Connect Provider to know which claims are used by the identity (sub claim).

Authentication	• **Upload OIDC RP License** Server license is used to enable general and product specific features. For OIDC RP license, refer to Upload OIDC RP License for more details. • **Configure User Auto Provisioning Module** The user auto provisioning module allows automatic provisioning of a newly created user's data to the default store or JDBC user store. The default user auto provisioning handler maps the value of the payload's claim/attribute name about an entity (typically the user) to AccessMatrix UAM, the OIDC Relying Party. Refer to Configure User Auto Provisioning Module for more details. • **Configure the Lookup Module** The lookup module is used to exchange the authorization code for OIDC token(s) to retrieve the user identity. Refer to Configure Lookup Module for the configuration details. • **Configure the Login Module** Login module provides a login or authentication method to verify user login credentials. Refer to Configure Login Module for the configuration details. • **Configure the Authentication Realm** Authentication realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to Configure Authentication Realm for more details. • **Assign Authentication Realm to the User** User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the Login Accounts tab. Refer to Assign Authentication Realm to User for more details.
Reauthentication	• **Upload OIDC RP License** Server license is used to enable general and product specific features. For OIDC RP license, refer to Upload OIDC RP License for more details. • **Configure OIDC RP Login Module for User Reauthentication** The OIDC RP login module is used to exchange the authorization code for OIDC token(s) to retrieve the user identity. Refer to Configure OIDC RP Login Module for User Reauthentication for the configuration details of the login module. • **Assign OIDC RP Login Module in USO Application Configuration** User reauthentication is currently supported by the AccessMatrix Common Login Page (CLP) for AccessMatrix Universal Sign-On (USO) applications. The configured OIDC RP login module must be assigned to the USO application in

	its Admin Console configurations page. Refer to *AccessMatrix Common Login Page Deployment Guide - Configure Reauthentication via External OIDC OP* for the configuration details.

Upload OIDC RP License

Before doing any configuration in the Admin Console, secure the server license required to implement AccessMatrix UAM as OIDC Relying Party. Ensure that OIDCRP license feature is added under **Social Network Login** in **UAM FEATURE** to enable OIDC RP support and to display:

- **Authentication Use Case** - "OIDC Relying Party Lookup Module" in the lookup module implementation list.
- **Reauthentication Use Case** - "OIDC Relying Party Login Module" in the login module implementation list.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the Server Id and click **Edit**.
3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

4.1. Configurations for Authentication

Configure User Auto Provisioning Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Auto Provisioning Module**.
2. Click the **Create** button and then select the "Default User Auto Provisioning Handler" implementation.
3. Click **Next** and enter the required values for the mapping of the payload's claim to AccessMatrix UAM that works as an OIDC Relying Party.
4. Enter a **User Auto Provisioning Module Id** (e.g. "AutoProv") and a **User Auto Provisioning Module Name** (e.g. "OIDC RP User Auto Prov").
5. Specify the **User Store** value that is associated with the User Auto Provisioning Module.
6. Specify the **Mapping User Id** value that indicates the payload's claim/attribute name of an entity (typically the user) to map its value to AccessMatrix User Id, or enter "userIdentity" that indicates the selected claim value of identity mapping for OIDC Relying Party.
7. Specify the **Mapping User Name** value that indicates the payload's claim/attribute name of an entity (typically the user) to map its value to the AccessMatrix User Name, or enter "userIdentity", which indicates the selected claim value of identity mapping for OIDC Relying Party.
8. Click **Save**. A new user auto provisioning module is created.

You may refer to *UAM Administration Guide - User Auto Provisioning Module Configuration* for detailed information about the module.

> **Note:**
>
> - User auto provisioning is only supported for default store and JDBC user store.
> - For user auto provisioning to be enabled, the **User Mapping Lookup Option** attribute must be set to "Identity Mapping". Refer to the Configure Lookup Module section to set this attribute.

User Auto Provisioning Example Use Case

For OpenID Connect Relying Party lookup module, most of the claims inside the ID token can be used for mapping during user auto-provisioning. These claims are used as the auto-provisioning payload; then it

can be mapped as configured in the Auto Provisioning Module. For example, the "sub" claim can be used to map to UserId. If OpenID Provider includes a claim for the user's name, then it can be mapped for the User Name.

"userIdentity" payload is a special payload that will contain the user identity as configured in the OIDC Relying Party lookup module. If in the lookup module, the identity is configured to use "sub", then the "userIdentity" payload will have the same content as the "sub" claim. In SingPass Mobile use case, where there is a claim processing. The "userIdentity" payload will contain the after-processing. Thus, it may contain the SingPass NRIC or SingPass UUID based on the OIDC Relying Party lookup module configuration.

OpenID Provider	Mapping User Id	Mapping User Name
Google	email	name
SingPass Mobile	userIdentity	userIdentity or provided by parameter during API call

Configure Lookup Module

To create an OIDC Relying Party lookup module, perform the following steps:

1. Navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane.
2. Click the **Create** button and select "OIDC Relying Party Lookup Module" implementation from the **Choose User Lookup Module Implementation** page.
3. Click **Next**.
4. Select "Yes" under the **PKCE Flow is Used** attribute if the Authorization Code flow with PKCE is used. "No" is selected by default.

   > **Note:** It is highly recommended that the Authorization Code flow with PKCE is used.

5. Provide the necessary values for the remaining lookup module attributes.
6. Save the changes.

> **Note:** ' * ' denotes a mandatory attribute.

Attribute Name	Description
*User Lookup Module Id	Unique ID of user lookup module, e.g. "OIDCRPLM'.
*User Lookup Module Name	Name of user lookup module, e.g. "OIDC RP Lookup Module".
Description	Description of user lookup module.
Version	User lookup module entry version.
Implementation	Type of user lookup module implementation, i.e. "OIDC Relying Party Lookup Module (For Authorization Code flow)".
User Store	User store associated with the user lookup module.
Attributes Tab	
Token Endpoint Authentication Method	Relying Party must indicate which method to use in order to authenticate itself to the OpenID Provider's token endpoint. Options: • "ClientSecret" - This authentication method requires client secret to be configured for the client authentication. • "PrivateKeyJWT" - This authentication method requires signing key pair to be generated in this module, so that it can be used to sign a JWT as JWT Assertion for the client authentication. Default: ClientSecret
Use OpenID Provider Token Endpoint As Audience	Specifies what will be used as the value for "aud" in the payload of the JWT when generating the Client Assertion JWT for the "PrivateKeyJWT" **Token Endpoint Authentication Method**. • If set to "true", the OpenID Provider's token endpoint URL is used as the "aud" value. • If set to "false", the OpenID Provider Issuer URL is used as the "aud" value. Default: false
*Client Id	A Client Identifier issued by the OpenID Provider during client registration. It is a unique string representing the client.
Client Secret	Client credential of a client used for authenticating with the authorization server. It is typically provided by the OpenID Provider. Providing a value for this is required if **Token Endpoint Authentication Method** is set to "ClientSecret".
Wrapping Key	Wrapping key is required to generate OpenID Connect encryption key pair and signing key pair. This key is used to protect system-generated encryption key pair and signing key pair.

Attribute Name	Description
PKCE Flow is Used	This setting determines if the Authorization Code flow with PKCE is used. Using the PKCE flow is strongly recommended. Default: false
OpenID Provider	
Notes: The lookup module will use the Authorization Endpoint and Token Endpoint in this precedence: 1. The configured **Authorization Endpoint** and **Token Endpoint**. 2. The Authorization Endpoint and Token Endpoint values acquired from the OpenID Provider Configuration URL metadata. OpenID Provider Signing Key used will be in this precedence: 1. The configured **OpenID Provider Signing Keys** values. 2. The configured **JWKS Endpoint**. 3. The JWKS URL value acquired from the OpenID Provider Configuration URL metadata.	
OpenID Provider Configuration URL	The configuration URL of the OpenID Provider that specifies the metadata and configuration.
OpenID Provider Issuer	Issuer identification of the OpenID Provider.
Authorization Endpoint	URL of the authorization server's authorization endpoint.
Token Endpoint	URL of the authorization server's token endpoint. **Note:** Only specify the value if "ClientSecret" is the token endpoint authentication method.
JWKS Endpoint	URL of JSON Web Key Set (JWKS) endpoint.
OpenID Provider Signing Keys	The token signing keys in the form of JSON Web Key (JWK) set or X509 Public Key string, published by the OpenID Provider for ID token signature verification. Example of OpenID JWKs in JSON: {"keys":[{"kty":"RSA","e":"AQAB","use":"sig","kid":"OAS-LLF-eSOLmmc","alg":"RS256","n":"puIJByYT3pXnrQsLI_oH...truncated...XwZWZkiqwXhJORQ"}]}
Default Values of Authentication Request for Authorization Endpoint	

Attribute Name	Description
Note: When preauthenticate method is called, these default values will be used to compute the Authentication Request Query URL for the application to call OIDC Authorization Endpoint.	
Scope	Specifies the scope value that the authorization server supports. It determines the behavior of the OIDC flow. Default: openid
Response Type	Specifies the response-type value that the OpenID Provider supports. It determines the authorization processing flow to be used by the OpenID Provider. Default: code
*Redirection URI	Redirection URI sent by Relying Party as part of an authentication request. This Redirection URI will be the Relying Party endpoint to receive authorization code sent by OpenID Provider, and must also be registered in the OpenID Provider configuration.
User Identity Mapping	
This contains configurations to map the ID token's claim to AccessMatrix user. Based on the **ID Token's Claim Name**, this lookup module retrieves the value from ID token as the identity value. This value will then mapped to AccessMatrix user based on the **User Mapping Lookup Option**. For example, if you want to use "email" claim as the AccessMatrix user's Id, you can specify "email" in the **ID Token's Claim Name** attribute and "User Id" for the **User Mapping Lookup Option** attribute. If there is a need to have further processing of the claim value to get the identity value, the **ID Token's Claim Value Processing** or **Custom ID Token's Claim Value Processing** can be configured. For Singapore SingPass, "sub" claim contains the user identity, but it requires further processing to extract the SingPass NRIC or SingPass UUID. In this case, you need to specify "SingPass NRIC" or SingPass UUID" for the **ID Token's Claim Value Processing** attribute. For custom implementation, a custom class can be registered via a plugin and then you may specify the short class name in **Custom ID Token's Claim Value Processing**.	
ID Token's Claim Name	Specify the claim name of the ID token to be used for user mapping. Default: sub
User Mapping Lookup Option	Map **ID Token's Claim Name** to AccessMatrix user by user attribute, user field or identity mapping. "Identity Mapping" is used to store external identity. If **Auto Create New User** is set to true, "Identity Mapping" must be selected. Options: • "User UUID"

Attribute Name	Description
	• "User Id" • "Identity Mapping" Default: User UUID
ID Token's Claim Value Processing	This process user's ID Token to retrieve SingPass user identity based on the selected option that is either SingPass UUID or SingPass NRIC. The value of SingPass user identity is used to map with AccessMatrix user based on the **User Mapping Lookup Option** setting. Default: None
Custom ID Token's Claim Value Processing	Specify a class short name that was implemented to have custom handling to resolve user identity for User Identity Mapping.
Resource Retrieval	
Enable Auto Retrieval from Resource Endpoint	This setting is to enable calling to resource endpoint to retrieve user entitlement and role and third party authorization relationship. Default: false
Resource Endpoint	URL to retrieve user entitlement and role and third party authorization relationship.
Resource Endpoint Response Format	The response format of the resource endpoint. Default: JWT
Advanced	
Nonce and State Expiry (in minutes)	This setting is used to compute the expiry time for both the nonce and state. This expiry time is returned to the application together with the newly generated nonce, state, and the computed Authentication Request Query URL. Value should be from 5 to 30. Default: 10
Include OIDC Payload Data in Returned Result	This setting enables the OIDC payload data to be returned to the API caller. When set to "true", this returns the following attributes along with the ID token claims: • id_token • access_token • resource_info • refresh_token • user_auto_created

Attribute Name	Description
	Note: • The user_identity attribute is always returned to the API caller, regardless of whether this attribute is set to "true" or "false". • If the OIDC RP Servlet is used, this setting should be set to "false". Enable this when OIDC RP Servlet is not being used and the RP may need to use the returned OIDC tokens to call resources or to manage the RP application session, etc. Default: false
Endpoint Response Content Length Validation Limit	Specifies a new content-length validation limit for the endpoint response. If the content-length exceeds this limit, the response will not be parsed and an error will be thrown. This attribute is used to allow endpoint responses with content-length that is larger than what a typical request response would have. To remove the validation check on content-length, set a negative value (e.g. -1). **Note:** To prevent abuse or DoS attacks, the validation check on content-length of response is set to 16384 (bytes) by default.
JWE Tab	
OpenID Provider may apply encryption to ID Token using JWE. This requires the Relying Party to provide an encryption key (public key part of a key pair). The OIDC Relying Party lookup module allows system-generated key pair or key pair stored in a keystore. With system-generated key pair, AccessMatrix generates the key pair and stores it in the database. Backed by AccessMatrix Key Management, when the key pair is stored in the database, the key pair is encrypted by another key ("Wrapping Key"), which can optionally be an HSM key for a better security. AccessMatrix also reads the key pair from a keystore (JKS or PKCS12) to support a key pair generated outside of AccessMatrix. Various key rollover strategies are supported. Refer to the Generate/ Rollover Encryption Key Pair section for more details.	
Key Rollover Strategy	Specifies the key rollover strategy used. • The “From system-generated key to system-generated key” strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. • The "From system-generated key to keystore" strategy allows a keystore key pair to be set as the active encryption key pair. **Note:** The **Rollover Key Pair For JWE** button will be disabled after the key rollover to the keystore key pair has been completed.

Attribute Name	Description
	• The "From keystore to keystore" strategy allows a new keystore key pair to replace an existing keystore key pair as the active signing key pair. Default: From system-generated key to system-generated key
ID Token is JWE Encrypted	Setting to true indicates that the ID token is JWE (JSON Web Encryption) encrypted. Default: false
Expiry of Old Key (in hours)	Expiry time of the old key is computed based on the creation time of the new encryption key pair (after key rollover) + the hour(s) configured in this setting. The value should not be less than 1. Default: 3
System-Generated Key Pair	
Encryption Key Algorithm	Algorithm used to generate the key pair for encryption. Options: • "EC-P256" • "EC-P384" • "EC-P521" • "RSA-2048" • "RSA-4096"
Key Pair in Keystore (Legacy)	
Keystore	Specifies the file path of the Java keystore file to be used. The keystore keys must be generated using a supported algorithm. Refer to the **Encryption Key Algorithm** attribute for the list of supported algorithms.
Keystore Passphrase	The passphrase of the Java keystore.
New Key Alias	Alias of the new key pair, as listed in the keystore, that will be set as the active key pair upon clicking the **Key Rollover for JWE** or **Load Initial Key Pair for JWE** buttons.
Old Key Alias	Alias of the existing key pair to be replaced, as listed in the keystore.
JWS Tab	
The OpenID Provider may require the Relying Party to call the Token endpoint using a Private Key JWT as its client authentication method. In this case, the Relying Party needs to generate a JWT token instead of using client_secret. The OIDC Relying Party lookup module allow system-generated key pair to be used to sign the JWT token.	

Attribute Name	Description
With system-generated key pair, AccessMatrix generates the key pair and stores it in the database. Backed by AccessMatrix Key Management, when the key pair is stored in the database, the key pair is encrypted by another key ("Wrapping Key"), which can optionally be an HSM key for a better security. AccessMatrix also reads the key pair from a keystore (JKS or PKCS12) to support a key pair generated outside of AccessMatrix. Various key rollover strategies are supported. Refer to the Generate/ Rollover Signing Key Pair section for more details.	
Key Rollover Strategy	Specifies the key rollover strategy used. • The “From system-generated key to system-generated key” strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. • The "From system-generated key to keystore" strategy allows a keystore key pair to be set as the active signing key pair. **Note:** The **Rollover Key Pair For JWS** button will be disabled after the key rollover to the keystore key pair has been completed. • The "From keystore to keystore" strategy allows a new keystore key pair to replace an existing keystore key pair as the active signing key pair. Default: From system-generated key to system-generated key
System-Generated Key Pair	
Signing Key Algorithm	Algorithm used to generate the key pair for signing. Options: • "EC-P256" • "EC-P384" • "EC-P521"
Key Pair in Keystore (Legacy)	
Keystore	Specifies the file path of the Java keystore file to be used. The keystore keys must be generated using a supported algorithm. Refer to the **Signing Key Algorithm** attribute for the list of supported algorithms.
Keystore Passphrase	The passphrase of the Java keystore.
New Key Alias	Alias of the new key pair, as listed in the keystore, that will be set as the active key pair upon clicking the **Key Rollover for JWS** or **Load Initial Key Pair for JWS** buttons.
Old Key Alias	Alias of the existing key pair to be replaced, as listed in the keystore.

Attribute Name	Description
Client Authentication using Private Key JWT	
Expiry (in minutes)	This value is used to compute expiry time of the JWT Assertion. (uneditable) Default: 2
Delay Use of New Key (in hours)	This value determines the delay time when the system can start using the new generated key to sign the Client Assertion JWT if the old key is present. This is to allow the OpenID Provider to retrieve the new key before the key starts being used. The value should be from 1 to 240. Default: 6
Proxy Tab	
HTTP Forward Proxy	
Enabled	Setting to true will enable the HTTP Proxy. Default: false
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.
User Name	Refers to the username used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If **NTLM Authentication** is enabled, **Domain Name** must be specified. In addition to that, it might be necessary to configure the AccessMatrix server host domain name or IP address in **amsystem.properties** with the property key "am.social.ntlm.host" for each instance.
Domain Name	Refers to the domain name of AD used in NTLM authentication.
HTTP Reverse Proxy	
Replace OpenID Provider Configuration Host Name	The OpenID Provider Configuration URL will be triggered during runtime. By default, the hostname of the **OpenID Provider Configuration URL** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
Replace OpenID Provider Token	The OpenID Provider Token Endpoint will be triggered during runtime. By default, the hostname of the **OpenID Provider Token Endpoint** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.

Attribute Name	Description
Endpoint Host Name	
Replace OpenID Provider JSON Web Key (JWK) URI Host Name	The OpenID Provider JSON Web Key (JWK) URI will be triggered during runtime. By default, the hostname of the **OpenID Provider JSON Web Key (JWK) URI** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
Replace Resource Endpoint Host Name	The Resource Endpoint URL will be triggered during runtime. By default, the hostname of the **Resource Endpoint** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
OIDC RP Servlet Tab	
Landing Page URL	The target URL to go to after the user has been successfully authenticated/authorized via OIDC flow.
Error Page URL	Error page URL of relying party's application that should process the returned error response and display the corresponding error page.
Landing Page URL Protection	
Prefix for URL Protection	The prefix that will be appended to the URL requiring protection. It is an OIDC RP initialization URL in the following format: https://<hostname:port>/clp/rp/init/<realm_Id>. **Note:** The URL that requires protection must be encoded in Base64 format.
User Auto Provisioning Tab	
This User Auto Provisioning module allows an automatic creation of a user when a user is not found in the system according to the identity mapping. The auto-provisioning module maps data acquired from OIDC claims to the user information.	
Auto Create New User	Select "true" to allow creating a new user automatically if a user is not found. If set to "true", then **User Mapping Lookup Option** must be set to "Identity Mapping". **Note:** This feature is only supported for the default store and JDBC user store. Default: false
User Auto Provisioning Module	Select a module to automatically support user provisioning to the default store or JDBC user store. Select Configure User Auto Provisioning Module for more details.
Session Management Tab	

Attribute Name	Description
Persist OIDC Tokens	Select "true" to enable the caching of OIDC tokens. This is used for the persistence of user sessions. Default: false
OIDC Tokens to be Cached [Hidden]	This attribute determines which OIDC tokens are cached. Do note that if the OIDC RP Servlet is used in your deployment, only the refresh token must be persisted. The following configurations are available: • "Refresh Token" - if this is enabled, only the refresh token and the expiry of the access token are cached. **Note:** The **Self-introspect Access Token** attribute must be set to “true” to enable OIDC token refresh upon calling the AM5 session refresh API. • "Access Token + Refresh Token" - if this is enabled, both the refresh token and the access token are cached. Default: Refresh Token
Token Refresh	
Allow Token Refresh	This attribute allows OIDC tokens to be refreshed upon calling the AM5 session refresh API. Default: false
Refresh Token Lifetime/Expiry (hours)	This attribute determines the validity period of a refresh token, such that: • Refresh token expiry = refresh token generation date + **Refresh Token Lifetime/Expiry (hours)** attribute value **Note:** This value should be provided by the OP based on their requirements for the housekeeping of the persisted tokens. Default: 168 (7 days) Value range is 1 (1 hour) to 8,760 (1 year).
Self-introspect Access Token	When set to "true", if a token refresh request is triggered, self-introspection will first be performed on the persisted access token. The refresh token endpoint will only be called if the introspected access token has expired. **Note:** The **Persist OIDC Tokens** attribute must be set to "true" for this feature to be enabled. Default: false

Configure Session Management

This section details the steps needed to enable the caching and persistence of refresh tokens and/or access tokens obtained from the OIDC OP. Once enabled, the following session management features can be used:

- **Session Refresh API**

  authn/session/refresh

Calling the AM5 Session Refresh API refreshes (and optionally, introspects) the access token provided by the OIDC OP. This is used to extend a user session and maintain a user session across multiple applications. If the OIDC token refresh process with the OIDC OP has indicated that the refresh token is expired, invalid, expired or revoked (where an invalid_grant OAuth error is returned), the cached OIDC token and its AM5 session will be removed.

- **Logout API**

  authn/session/persistent/federated/logout Calling the Logout API invalidates all of the user's persisted AM5 session(s), and invalidates all their federated session(s) by removing the cached OIDC token(s). This is used to logout of user session(s) of all applications sharing the same user ID from within the specified user store (and optionally, from within the specified segment as well).

  > **Note:** Expired refresh tokens will be purged on a daily basis during off peak hours to have the corresponding cached OIDC token and its AM5 session removed.

Enable Session Management Features

Perform the following steps to enable OIDC token persistence and the above features:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and select the previously created OIDC RP lookup module.
2. Under the **Session Management** tab, select "true" for the **Persist OIDC Tokens** attribute.
3. Under the **Token Refresh** header, select "true" for the **Allow Token Refresh** attribute.
4. Enter a suitable value under **Refresh Token Lifetime/Expiry (hours)**. This value can be obtained from the OP. The OIDC RP lookup module will use this for the housekeeping of the persisted tokens.
5. Select a suitable value for the **Self-introspect Access Token** attribute:
   - **Selecting "true"** - When the AM5 Session Refresh API is called, the access token's expiry is validated before it is refreshed. If it has not expired, the token will not be refreshed.
   - **Selecting "false"** - When the AM5 Session Refresh API is called, the access token's expiry will not be validated and the token will always be refreshed.

     > **Note**: The above scenarios assume that the **Persist OIDC Tokens** and **Allow Token Refresh** attributes are set to "true".
6. Save the changes.

Refer to the OIDC Relying Party lookup module attribute table under Configure Lookup Module for more information on the attributes detailed in the steps above.

Session Management Behavior

The session management behavior will vary depending on the session concurrency configurations in the **Session** tab in **Administration Dashboard > Configuration > Global Setting**. Assuming session management has been enabled according to the steps detailed above, below is an overview of how each session concurrency configuration affects token persistence upon user login:

- **Allow concurrent logins**
  After token exchange, OIDC token(s) will be persisted.
- **Reject new login**
  If within maximum number of allowed active sessions, then after token exchange, OIDC token(s) will be persisted.
- **Terminate user's oldest session to allow the new one**
  After token exchange, OIDC token(s) will be persisted. Token(s) of older session(s) will be removed.
- **Ask user's confirmation to terminate the oldest session to allow the new one**
  After token exchange, OIDC token(s) will first be saved as preactivated tokens. Once the user has confirmed that they wish to terminate the existing session, the preactivated tokens be changed to active tokens and the active tokens will be persisted. Token(s) of older session(s) will be removed.

Refer to the **Global Setting Page** attribute table in the *Common Administration Guide - System Configuration* section for more information on the attributes detailed in the steps above.

Other Configurations for OIDC RP Lookup Module

The OIDC RP lookup module and OIDC RP login module share many configurations. Refer to the Other OIDC RP Module Configurations section for detailed information on all of the advanced configurations shared between both modules.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.

2. Click **Create** and select the "Social Network Login" implementation from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "SocNetLM") and a **Login Module Name** (e.g. "Social Network Login Module").
4. Select the **Login Policy** and the **Authentication Level** values, if necessary.
5. Click **Save**. A new login module is created.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "RPRealm") and an **Authentication Realm Name** (e.g. "OIDC RP Realm").
4. Specify the **User Lookup Module** value that will be used to validate user identity, e.g. "OIDC RP Lookup Module (OIDCRPLM)".
5. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "Social Network Login Module (SocNetLM)".
6. Click **Save**. A new authentication realm is created.

Assign Authentication Realm to User

A user may belong to a default store or external user store. You may refer to *AccessMatrix Common Administration Guide - Manage User Stores* to know about the different types of user stores and how to configure it.

To create a new user in default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for OIDC Relying Party implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id, e.g. "user01".
*User Name	Name of user, e.g. "user01".

Attribute Name	Description
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in previous section [e.g. OIDC RP Realm (RPRealm)] and click **OK**. Both authentication realm and the login module(s) associated to it will be assigned to the user.	

Click **Save**. A new user is created.

You may refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

4.2. Configurations for Reauthentication

Configure OIDC RP Login Module for Reauthentication via External OIDC OP

User reauthentication via an external OIDC OP is supported by the AccessMatrix Common Login Page (CLP) for AccessMatrix Universal Sign-On (USO) applications. This authentication method uses the Authorization Code flow or the Authorization Code flow with PKCE.

The OIDC RP login module must be configured as part of the requirements for enabling this feature. To configure the login module, perform the following:

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "OIDC Relying Party Login Module" implementation from the **Choose Login Module Implementation** page.
3. Click **Next**.
4. Select "Yes" under the **PKCE Flow is Used** attribute if the Authorization Code flow with PKCE is used. "No" is selected by default.

   > **Note**: It is highly recommended that the Authorization Code flow with PKCE is used.

5. Enter a **Login Module Id** (e.g. "OIDCRPLogin") and a **Login Module Name** (e.g. "OIDC Relying Party Login Module").
6. Select the **Login Policy** and the **Authentication Level** values, if necessary.
7. Click **Save**. A new login module is created.

Attribute Name	Description
*Login Module Id	Unique ID of login module, e.g. "OIDCRPLogin"
*User Login Module Name	Name of login module, e.g. "OIDC RP Login Module".
Description	Description of login module.
Implementation	Type of login module implementation, i.e. "OIDC Relying Party Login Module".
Handler	Handler of login module, i.e. "OIDCRelyingPartyLoginModule".

Attribute Name	Description
Attributes Tab	
Token Endpoint Authentication Method	Relying Party must indicate which method to use in order to authenticate itself to the OpenID Provider's token endpoint. Options: • "ClientSecret" authentication method requires client secret to be configured for the client authentication. • "PrivateKeyJWT" authentication method requires signing key pair to be generated in this module, so that it can be used to sign a JWT as JWT Assertion for the client authentication. Default: ClientSecret
*Client Id	A Client Identifier issued by the OpenID Provider during client registration. It is a unique string representing the client.
Client Secret	Client credential of a client used for authenticating with the authorization server. It is typically provided by the OpenID Provider. Providing a value for this is required if **Token Endpoint Authentication Method** is set to "ClientSecret".
Wrapping Key	Wrapping key is required to generate OpenID Connect encryption key pair and signing key pair. This key is used to protect system-generated encryption key pair and signing key pair.
PKCE Flow is Used	This setting determines if the Authorization Code flow with PKCE is used. Using the PKCE flow is strongly recommended. Default: false
OpenID Provider	
Notes: The login module will use the Authorization Endpoint and Token Endpoint in this precedence: 1. The configured **Authorization Endpoint** and **Token Endpoint**. 2. The Authorization Endpoint and Token Endpoint values acquired from OpenID Provider Configuration URL metadata. OpenID Provider Signing Key used will be in this precedence: 1. The configured **OpenID Provider Signing Keys** values. 2. The configured **JWKS Endpoint**. 3. The JWKS URL value acquired from the OpenID Provider Configuration URL metadata.	
OpenID Provider Configuration URL	The configuration URL of the OpenID Provider that specifies the metadata and configuration.

Attribute Name	Description
OpenID Provider Issuer	Issuer identification of the OpenID Provider.
Authorization Endpoint	URL of the authorization server's authorization endpoint.
Token Endpoint	URL of the authorization server's token endpoint. **Note:** Only specify the value if "ClientSecret" is the token endpoint authentication method.
JWKS Endpoint	URL of JSON Web Key Set (JWKS) endpoint.
OpenID Provider Signing Keys	The token signing keys in the form of JSON Web Key (JWK) set or X509 Public Key string, published by the OpenID Provider for ID token signature verification. Example of OpenID JWKs in JSON: {"keys":[{"kty":"RSA","e":"AQAB","use":"sig","kid":"OAS-LLF-eSOLmmc","alg":"RS256","n":"puIJByYT3pXnrQsLI_oH...truncated...XwZWZkiqwXhJORQ"}]}
Default Values of Authentication Request for Authorization Endpoint	
Note: When preauthenticate method is called, these default values will be used to compute the Authentication Request Query URL for the application to call OIDC Authorization Endpoint.	
Scope	Specifies the scope value that the authorization server supports. It determines the behavior of the OIDC flow. Default: openid
Response Type	Specifies the response-type value that the OpenID Provider supports. It determines the authorization processing flow to be used by the OpenID Provider. Default: code
Redirection URI	Redirection URI sent by Relying Party as part of an authentication request. This Redirection URI will be the Relying Party endpoint to receive authorization code sent by OpenID Provider, and must also be registered in the OpenID Provider configuration.
Trigger Force Login Option	Configures the optional 'prompt' parameter in authentication requests for the "Login" and/or "Reauthentication" use cases. Specifying one of these values causes 'prompt=login' to be included in the request for that use case. This means that the user will be prompted to log in even if the user's recent login session is still valid for that use case. Multiple values can be specified.

Attribute Name	Description
Login Hint Mapping Option	Configures the optional 'login_hint' parameter in the authentication request for the reauthentication use case only. Specifying a user identifier ("User Id" or "Email") causes it to be included in the request as the value of 'login_hint'. This can speed up the authentication process by supplying the user's identifier (i.e. their Id or email) in the login prompt to the OP.
User Identity Mapping	
This contains configurations to map the ID token's claim to AccessMatrix user. Based on the **ID Token's Claim Name**, this login module retrieves the value from ID token as the identity value. This value will then mapped to AccessMatrix user based on the **User Mapping Lookup Option**. For example, if you want to use "email" claim as the AccessMatrix user's Id, you can specify "email" in the **ID Token's Claim Name** attribute and "User Id" for the **User Mapping Lookup Option** attribute. If there is a need to have further processing of the claim value to get the identity value, **ID Token's Claim Value Processing** or **Custom ID Token's Claim Value Processing** can be configured. For Singapore SingPass, "sub" claim contains the user identity, but it requires further processing to extract the SingPass NRIC or SingPass UUID. In this case, you need to specify "SingPass NRIC" or SingPass UUID" for the **ID Token's Claim Value Processing** attribute. For custom implementation, a custom class can be registered via a plugin and then you may specify the short class name in **Custom ID Token's Claim Value Processing**.	
ID Token's Claim Name	Specify the claim name of the ID token to be used for User Mapping. Default: sub
User Mapping Lookup Option	Map ID token's Claim to AccessMatrix user by user attribute, user field or identity mapping. "Identity Mapping" is used to store external identity. If **Auto Create Secondary Account** is set to "true", "Identity Mapping" must be selected. Options: • "User UUID" • "User Id" • "Identity Mapping" Default: User UUID
ID Token's Claim Value Processing	This process user's ID Token to retrieve SingPass user identity based on the selected option that is either SingPass UUID or SingPass NRIC. The value of SingPass user identity is used to map with AccessMatrix user based on the **User Mapping**

Attribute Name	Description
	Lookup Option setting. Default: None
Custom ID Token's Claim Value Processing	Specify a class short name that was implemented to have custom handling to resolve user identity for User Identity Mapping.
Resource Retrieval	
Enable Auto Retrieval from Resource Endpoint	This setting is to enable calling to resource endpoint to retrieve user entitlement and role and third party authorization relationship. Default: false
Resource Endpoint	URL to retrieve user entitlement and role and third party authorization relationship.
Resource Endpoint Response Format	The response format of the resource endpoint. Default: JWT
Advanced	
Nonce and State Expiry (in minutes)	This setting is used to compute the expiry time for both the nonce and state. This expiry time is returned to the application together with the newly generated nonce, state, and the computed Authentication Request Query URL. Value should be from 5 to 30. Default: 10
Include OIDC Payload Data in Returned Result	This setting enables the OIDC payload data to be returned to the API caller. When set to "true", this returns the following attributes along with the ID token claims: • id_token • access_token • resource_info • refresh_token • user_auto_created **Notes:** • The user_identity attribute is always returned to the API caller, regardless of whether this attribute is set to "true" or "false". • If the OIDC RP Servlet is used, this setting should be set to "false". Enable this when OIDC RP Servlet is not being used and the RP may need to use the returned OIDC tokens to call resources or to manage the RP application session, etc.

Attribute Name	Description
	Default: false
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect.
Authentication Level	Strength of authentication from 1 (lowest) to 10 (highest). Unused. Note: This is an attribute of the login module used for UAM product. Default: 1
JWE Tab	
OpenID Provider may apply encryption to ID Token using JWE. This requires the Relying Party to provide an encryption key (public key part of a key pair). The OIDC Relying Party login module allows the use of a system-generated key pair or a key pair stored in a keystore. With system-generated key pair, AccessMatrix generates the key pair and stores it in the database. Backed by AccessMatrix Key Management, when the key pair is stored in the database, the key pair is encrypted by another key ("Wrapping Key"), which can optionally be an HSM key for a better security. AccessMatrix also reads the key pair from a keystore (JKS or PKCS12) to support a key pair generated outside of AccessMatrix. Various key rollover strategies are supported. Refer to the Generate/ Rollover Encryption Key Pair section for more details.	
Key Rollover Strategy	Specifies the key rollover strategy used. • The “From system-generated key to system-generated key” strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. • The "From system-generated key to keystore" strategy allows a keystore key pair to be set as the active encryption key pair. **Note:** The **Rollover Key Pair For JWE** button will be disabled after the key rollover to the keystore key pair has been completed. • The "From keystore to keystore" strategy allows a new keystore key pair to replace an existing keystore key pair as the active signing key pair. Default: From system-generated key to system-generated key
ID Token is JWE Encrypted	Setting to "true" indicates that the ID token is JWE (JSON Web Encryption) encrypted. Default: false

Attribute Name	Description
Expiry of Old Key (in hours)	Expiry time of old key is computed based on the created time of the new encryption key pair (after key rollover) plus the hour(s) configured in this setting. Value should not be less than 1. Default: 3
System-Generated Key Pair	
Encryption Key Algorithm	Algorithm used to generate the key pair for encryption. Options: • "EC-P256" • "EC-P384" • "EC-P521" • "RSA-2048" • "RSA-4096"
Key Pair in Keystore (Legacy)	
Keystore	Specifies the file path of the Java keystore file to be used. The keystore keys must be generated using a supported algorithm. Refer to the Encryption Key Algorithm attribute for the list of supported algorithms.
Keystore Passphrase	The passphrase of the Java keystore.
New Key Alias	Alias of the new key pair, as listed in the keystore, that will be set as the active key pair upon clicking the **Key Rollover for JWE** or **Load Initial Key Pair for JWE** buttons.
Old Key Alias	Alias of the existing key pair to be replaced, as listed in the keystore.
JWS Tab	
The OpenID Provider may require the Relying Party to call the Token endpoint using a Private Key JWT as its client authentication method. In this case, the Relying Party needs to generate a JWT token instead of using client_secret. The OIDC Relying Party Login Module allows a system-generated key pair to be used to sign the JWT token. With system-generated key pair, AccessMatrix generates the key pair and stores it in the database. Backed by AccessMatrix Key Management, when the key pair is stored in the database, the key pair is encrypted by another key ("Wrapping Key"), which can optionally be an HSM key for a better security. AccessMatrix also reads the key pair from a keystore (JKS or PKCS12) to support a key pair generated outside of AccessMatrix.	

Attribute Name	Description
Various key rollover strategies are supported. Refer to the Generate/ Rollover Signing Key Pair section for more details.	
Key Rollover Strategy	Specifies the key rollover strategy used. • The “From system-generated key to system-generated key” strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. • The "From system-generated key to keystore" strategy allows a keystore key pair to be set as the active signing key pair. **Note:** The **Rollover Key Pair For JWS** button will be disabled after the key rollover to the keystore key pair has been completed. • The "From keystore to keystore" strategy allows a new keystore key pair to replace an existing keystore key pair as the active signing key pair. Default: From system-generated key to system-generated key
System-Generated Key Pair	
Signing Key Algorithm	Algorithm used to generate the key pair for signing. Options: • "EC-P256" • "EC-P384" • "EC-P521"
Key Pair in Keystore (Legacy)	
Keystore	Specifies the file path of the Java keystore file to be used. The keystore keys must be generated using a supported algorithm. Refer to the **Signing Key Algorithm** attribute for the list of supported algorithms.
Keystore Passphrase	The passphrase of the Java keystore.
New Key Alias	Alias of the new key pair, as listed in the keystore, that will be set as the active key pair upon clicking the **Key Rollover for JWS** or **Load Initial Key Pair for JWS** buttons.
Old Key Alias	Alias of the existing key pair to be replaced, as listed in the keystore.
Client Authentication using Private Key JWT	
Expiry (in minutes)	This value is used to compute expiry time of the JWT Assertion. (uneditable) Default: 2

Attribute Name	Description
Delay Use of New Key (in hours)	This value determines the delay time when the system can start using the new generated key to sign the Client Assertion JWT if the old key is present. This is to allow the OpenID Provider to retrieve the new key before the key starts being used. The value should be from 1 to 240. Default: 6
Proxy Tab	
HTTP Forward Proxy	
Enabled	Setting to "true" will enable the HTTP Proxy. Default: false
Proxy Host Name	Refers to the URL of the proxy server.
Port	Refers to the port of the proxy server.
User Name	Refers to the username used to authenticate the proxy server.
User Password	Refers to the user's password used to authenticate the proxy server.
NTLM Authentication	Indicates whether NTLM authentication is used. If **NTLM Authentication** is enabled, **Domain Name** must be specified. In addition to that, it might be necessary to configure the AccessMatrix server host domain name or IP address in **amsystem.properties** with the property key "am.social.ntlm.host" for each instance.
Domain Name	Refers to the domain name of AD used in NTLM authentication.
HTTP Reverse Proxy	
Replace OpenID Provider Configuration Host Name	The OpenID Provider Configuration URL will be triggered during runtime. By default, the hostname of the **OpenID Provider Configuration URL** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
Replace OpenID Provider Token Endpoint Host Name	The OpenID Provider Token Endpoint will be triggered during runtime. By default, the hostname of **OpenID Provider Token Endpoint** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
Replace OpenID Provider JSON	The OpenID Provider JSON Web Key (JWK) URI will be triggered during runtime. By default, the hostname of **OpenID Provider JSON Web Key (JWK) URI** attribute is

Attribute Name	Description
Web Key (JWK) URI Host Name	used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
Replace Resource Endpoint Host Name	The Resource Endpoint URL will be triggered during runtime. By default, the hostname of the **Resource Endpoint** attribute is used. If a different hostname is specified (e.g. http://myforwardProxyhost), then it will be used for HTTP forward proxy use.
OIDC RP Servlet Tab	
Landing Page URL	The target URL to go to after the user has been successfully authenticated/authorized via OIDC flow.
Error Page URL	Error page URL of relying party's application that should process the returned error response and display the corresponding error page.
User Auto Provisioning Tab	
This User Auto Provisioning module allows the automatic creation of a secondary account according to the identity mapping, if a secondary account doesn't exist. The auto-provisioning module maps data acquired from OIDC claims to the user information.	
Auto Create Secondary Account	This feature is used to support auto provisioning for user based on the payload of the asserted user obtained from external Identity Provider. A secondary account will be created and linked to this user, if not already done so. If set to "true", then **User Mapping Lookup Option** must be set to "Identity Mapping". Default: false

Enable Force Login

When this feature is enabled, the optional prompt parameter is included in the authentication request with the value "login". This causes the user to always be prompted to log in for login and/or reauthentication use cases, even if their recent login session is still valid.

For example, enabling force login for reauthentication makes it mandatory for the user to log in again when attempting to perform certain actions that require reauthentication, such as retrieving a forgotten password from the USO Self-Service Portal.

To enable the force login feature, set the **Trigger Force Login Option** attribute to "Login", "Reauthentication", or both, depending on which use case you wish to enforce this for. Multiple values can be selected by using Ctrl + Click.

Enable Login Hint

> **Note:** This feature is only applicable for reauthentication use cases.

When this feature is enabled, the optional login_hint parameter is included in the authentication request with the value of the user's identifier (their Id or email, depending on the configurations). This causes the user's identifier to be supplied at the reauthentication login prompt, allowing the user to skip the entering of their identifier and potentially shortening the authentication process for a smoother user experience.

To enable the login hint feature, set the **Login Hint Mapping Option** attribute to "User Id" or "Email", depending on which one the OIDC OP uses for logins.

Other Configurations for OIDC RP Login Module

The OIDC RP login module and OIDC RP lookup module share many configurations. Refer to the Other OIDC RP Module Configurations section for all of the advanced configurations shared between both modules.

Assign the Login Module in USO Application Configuration

Refer to *AccessMatrix Common Login Deployment Guide - Configure Reauthentication via External OIDC OP* for the configuration details.

4.3. Common Configurations

Other OIDC RP Module Configurations

The following section provides detailed information on the advanced configurations that can be made in both the OIDC RP login module and OIDC RP lookup module.

Configure Lookup/Login Module for Use with Microsoft Entra ID User Store

> **Note**: Skip this section if you are not using Microsoft Microsoft Entra ID as the OIDC OpenID Provider.

This section provides additional instructions on the necessary configurations for the OIDC RP lookup/login module if you are using it with an Microsoft Entra ID user store.

> **Note**: For the authentication use case, you should additionally refer to the Create Microsoft Entra ID User Store section under *Common Administration Guide - Manage User Stores* for more information before continuing with the steps below.

Make the following additional configurations:

Additional Configurations in Microsoft Entra ID User Store

1. In the Microsoft Microsoft Entra ID web portal, navigate to **Microsoft Entra ID > Manage > App registrations** and select your app. The app's **Overview** page is displayed.
2. On the left pane, under **Manage**, select **Authentication > Platform configurations > Add a platform**.
3. Under **Configure platforms**, click on the **Web** tile. The **Configure Web** window is displayed.
4. Perform the following steps:
    a. For authentication use cases:
        i. Under **Redirect URIs**, enter "https://<hostname:port>/clp/rp/cb/<realmId>".

        > **Note**: Replace "<hostname:port>" with your publicly accessible AccessMatrix Server URI, and replace "<realmId>" with the Id of the authentication realm configured in the Configure Authentication Realm section.

        ii. Click **Configure** to complete the platform configuration.

b. For reauthentication use cases:

i. Under **Redirect URIs**, enter "https://<hostname:port>/clp/rp/reauthn/cb/<loginModuleId>".

> **Note**: Replace "<hostname:port>" with your publicly accessible AccessMatrix Server URI, and replace "<loginModuleId>" with the Id of the authentication realm configured in the Configure OIDC RP Login Module section.

ii. Click **Configure** to complete the platform configuration.

5. Next, navigate to **Manage > API Permissions > Add a permission**. The **Request API permissions** window is displayed.
6. Perform the following steps:
   a. Select **Microsoft APIs > Microsoft Graph**.
   b. Under **Delegated permissions**, search for the "openid" permission and select it.
   c. Click **Add permissions**.
   d. Click **Grant admin consent**.

Additionally, if the OIDC ID token needs to return the Microsoft Entra ID user's email for identity mapping, perform the following steps:

1. Navigate to **Manage > Token Configuration > Add optional claim**. The **Add optional claim** window is displayed.
2. Under **Token type**, select the **ID** option.
3. Tick either the **email** or **upn** checkbox under **Claim**.

> **Note**: Either claim can be selected. However, as the **email** field is not a mandatory field during Microsoft Entra ID user creation, some users may not have added an email address. For user stores with such cases, choose **upn**, as users' UPNs can be used for the same purpose.

4. Click **Add**.

Finally, refer to the A. Microsoft Entra ID as OIDC OP appendix for additional information on using Microsoft Entra ID as the OIDC OP, such as how to enable Microsoft Entra ID passwordless phone sign-in for the registered app.

Additional Configurations for OIDC RP Lookup/Login Module

1. Create the corresponding OIDC RP module by following the instructions in the Configure Lookup Module or Configure OIDC RP Login Module section if you have not yet done so.
2. (For OIDC RP lookup module only) Select your Microsoft Entra ID user store under the **User Store** attribute.
3. Under the **Attributes** tab, these are the default values you should enter for the following attributes:
    a. **Client Id** - Enter the Application (client) ID from your app's Overview page in the Microsoft Entra ID App registrations portal.
    b. **Client Secret** - Enter the client secret you generated for your app in the Microsoft Entra ID App registrations portal.
    c. **OpenID Provider Configuration URL** - Enter the OpenID Connect metadata document URI from your app's Overview page in the Microsoft Entra ID App registrations portal.

    > **Note**: Refer to *Common Administration Guide - Create Microsoft Entra ID User Store* if you are unable to locate the values referenced in the above steps.

    d. **Redirection URI** - Enter "https://<hostname:port>/clp/rp/cb/<realmId>".

    > **Note**: Replace "<hostname:port>" with your publicly accessible AccessMatrix Server URI, and replace "<realmId>" with the ID of the authentication realm configured in the Configure Authentication Realm section.

    e. **ID Token's claim Name** - Enter "email" or "upn", depending on which claim was selected in the previous sub-section.
    f. **User Mapping Lookup Option** - Enter "User Id".
4. Configure the remaining attributes as needed and save the changes.

Test OpenID Provider Configuration URL

Click the **Test OpenID Provider Configuration URL** button to test the module's connection to the configured OpenID Provider (OP) Configuration endpoint.

It also tests the connection using forward proxy and/or reverse proxy if configured.

Generate/ Rollover Encryption Key Pair

Click the **Generate Key Pair for JWE** button to generate an encryption key pair. A new key pair will be generated and set as the active key pair for ID Token encryption.

If a key pair already exists, the **Rollover Key Pair for JWE** button will be displayed instead. Click this button to perform a key rollover.

> **Important Note**: For keystores, only keystore key pairs generated with supported algorithms can be used as encryption key pairs. Supported algorithms:
>
> - EC-P256
> - EC-P384
> - EC-P521
> - RSA-2048
> - RSA-4096

Key Rollover Strategy

The key rollover strategy is determined using the **Key Rollover Strategy** attribute. The following options are available:

- **From system-generated key to system-generated key**
  This strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. This strategy is selected by default.
  There are several possible scenarios when this strategy is selected:
  - **System-generated key pair is not available** - The key pair from the keystore will be used instead. This keystore key pair is rolled over to a system-generated key pair.
  - **System-generated key pair is currently used** - The key rollover is performed normally. The old private key will be kept for decryption by the OIDC Relying Party. The use of this old private key for decryption will be allowed until the duration specified in the **Expiry of old key (in hours)** attribute has passed.
  - **Both system-generated key pair and keystore key pair are currently used** - The JWKS endpoint publishes the public key of the active system-generated key pair only. AccessMatrix uses both the system generated key pair and the key pair in the keystore. This is to allow a key rollover in the scenario where a keystore key pair was initially used and you wish to rollover to the system-generated key pair. The keystore key pair is not subjected to the **Expiry of old key (in hours)** attribute setting. The keystore key pair will be usable for decryption until it is removed from the OIDC RP module configuration.
- **From system-generated key to keystore**
  The keystore key pair will be regarded as the new (active) key pair and the existing system-generated key pair will be regarded as the old (disabled) key pair.

> **Note**: The **Rollover Key Pair for JWE** button will be disabled once the key rollover to the keystore key pair has been completed.

There are several possible scenarios when this strategy is selected:

- **System-generated key pair is not available** - The **Generate Key Pair for JWE** button is displayed under the **JWE** tab of the **User Lookup Module** page (authentication) or the **Login Module** (reauthentication) page. This must first be clicked for the system-generated key pair to be created. The button will then display the **Rollover Key Pair for JWE** label. Clicking this will perform the key rollover to the keystore key pair.
- **System-generated key pair is currently used** - The key rollover is performed normally. The old private key will be kept for decryption by the OIDC Relying Party. The use of this old private key for decryption will be allowed until the duration specified in the **Expiry of old key (in hours)** attribute has passed.

Generate/ Rollover Signing Key Pair

Click the **Generate Key Pair for JWS** button to create a system-generated key pair and set it as the active key for signing. Alternatively, if the "From keystore to keystore" strategy is selected, click the **Load Initial Key Pair for JWS** button to load a keystore signing key pair.

If a key pair already exists, the **Rollover Key Pair for JWS** button will be displayed instead. Click this button to perform a key rollover.

> **Important Note**: For keystores, only keystore key pairs generated with supported algorithms can be used as signing key pairs. Supported algorithms:
> - EC-P256
> - EC-P384
> - EC-P521

Key Rollover Strategy

The key rollover strategy is determined by the value specified in the **Key Rollover Strategy** attribute. The following options are available:

- **From system-generated key to system-generated key**

  This strategy maintains the existing key rollover behavior for customers who have just upgraded from an older version of AM5. If a system-generated key is not available, the key pair from the

keystore will be used instead. This strategy is selected by default.

There are several possible scenarios when this strategy is selected:

- **System-generated key pair is not available** - The key pair from the keystore will be used instead. This keystore key pair is rolled over to a system-generated key pair.
- **System-generated key pair is currently used** - The key rollover is performed normally. Once the key rollover is complete, the public keys of both the new and old key pairs are published via the JWKS endpoint for the OpenID Provider to verify the signature in the JWT.

- **From system-generated key to keystore**

  The keystore key pair will be regarded as the new (active) key pair and the existing system-generated key pair will be regarded as the old (disabled) key pair.

  > **Note**: The **Rollover Key Pair for JWS** button will be disabled once the key rollover to the keystore key pair has been completed.

  There are several possible scenarios when this strategy is selected:

  - **System-generated key pair is not available** - The **Generate Key Pair for JWS** button is first displayed under the **JWS** tab of the **User Lookup Module page** (authentication) or the **Login Module** (reauthentication) page. This must first be clicked for the system-generated key pair to be created. The button will then display the **Rollover Key Pair for JWS** label. Clicking this will perform the key rollover to the keystore key pair.
  - **System-generated key pair is currently used** - The key rollover is performed normally.

- **From keystore to keystore**

  The keystore key pair associated with the alias specified in the **New Key Alias** attribute will be regarded as the new (active) key pair. The existing keystore key pair associated with the alias specified in the **Old Key Alias** attribute will be regarded as the old (disabled) key pair.

  There are several possible scenarios when this strategy is selected:

  - **Keystore key pair is not available** - The **Load Initial Key Pair for JWS** button is first displayed under the **JWS** tab of the **User Lookup Module** page (authentication) or the **Login Module** (reauthentication) page. Clicking this sets the key pair associated with the alias in the **New Key Alias** attribute as the active signing key pair. The **Old Key Alias** attribute is left empty.

    Once completed, the button will then display the **Rollover Key Pair for JWS** label. To rollover to another new key pair, the alias of the existing key pair must be specified in the **Old Key Alias** attribute, while the alias of the new key pair must be specified in the **New**

Key Alias attribute. Clicking the button performs the key rollover to the **New Key Alias** key pair.

This behavior is illustrated in the table below:

Button	New Key Alias	Old Key Alias	Result
Load Initial Key Pair for JWS	test-key-a		"test-key-a" key pair is set as the new (active) signing key pair.
Rollover Key Pair for JWS	test-key-b	test-key-a	"test-key-a" key pair is set as the old (disabled) key pair. "test-key-b" key pair is set as the new (active) signing key pair.

- **Keystore key pair is currently used** - The key rollover is performed normally.

> **Note**: The key pair associated with the alias specified in the **Old Key Alias** attribute must match the existing signing key pair for the rollover to be successful.

The onboarding time of the new key pair can be configured by the **Delay Use of New Key (in hours)** attribute. This attribute's value determines the time delay before the system can start using the new key pair to sign the Client Assertion JWT if the old key is still present. The time delay ensures that the OpenID Provider has sufficient time to retrieve the new key before it starts being used.

View OIDC Relying Party JWK

Click **View OIDC Relying Party JWK** button to view the public key(s) published by this OIDC Relying Party. Upon clicking, a pop-up window will be opened. You may need to configure your web browser to allow pop-up windows.

The purpose of the JWK(s) can be identified with the value of the "use" claim:

- "sign" (for signature verification)
- "enc" (for encryption)

Enable Client Authentication Using Client Assertion JWT

OIDC RP supports Client Assertion JWT as one of the client authentication methods when calling the /token endpoint. To enable this feature, perform the following steps:

1. Create a wrapping key. This key is required to generate OIDC signing key pair. You may refer to *Common Administration Guide - Configure Encryption Key* to create a wrapper key.
2. Then, in the OIDC RP lookup/login module, select the previously created wrapper key and assign as the **Wrapping Key** value.
3. Set the value of the **Token Endpoint Authentication Method** attribute to "PrivateKeyJWT".
4. (Optional, for Authentication use case only) Specify the desired value for the **Use OpenID Provider Token Endpoint As Audience** attribute in the OIDC Relying Party lookup module:
    a. If set to "true", the OpenID Provider's token endpoint URL is used as the "aud" value in the payload of the JWT. This option should only be used if the OP allows the use of the OP token endpoint as an acceptable value for "aud" for the validation of the Client Assertion JWT.
    b. If set to "false", the OpenID Provider Issuer URL is used as the "aud" value in the payload of the JWT. This is the default value.
5. Select the **JWS** tab, and assign the algorithm used to generate the key pair in the **Signing Key Algorithm** attribute.
6. Save the changes.
7. Then, click the **Generate Signing Key Pair** button to generate the signing key pair.
   This generated signing key will be used to sign the Client Assertion JWT.

> **Note**: Creating a Client Assertion JWT for every call to the token endpoint is expensive. Therefore, OIDC RP reuses this generated Client Assertion JWT before it expires to enhance the efficiency and performance. The expiry time is set to 2 minutes by default as shown in the **Expiry (in minutes)** attribute.

Configure to Retrieve Resource Info using Access Token as Bearer Token

This configuration is used to retrieve the corresponding resource info from Resource Endpoint. This is done with the access token which was obtained from the /token endpoint. If the **Resource Endpoint Response Format** is set to "JWT", the JWT response would be validated using the key obtained from **JWKS Endpoint** or **OpenID Provider Signing Keys**.

1. In the OIDC RP lookup/login module's **Attributes** tab, look for the **Resource Retrieval** category.
2. Choose "Yes" for the **Enable Auto Retrieval from Resource Endpoint** attribute. Default is "No".
3. Configure the **Resource Endpoint URL** to retrieve the corresponding resource information.

4. Select the correct response format from **Resource Endpoint Response Format**. Default is "JWT".
5. (Optional) if HTTP Reverse Proxy is used, navigate to the **Proxy** tab, specify the reverse proxy hostname in **Replace Resource Endpoint Host Name**.

Configure to Include OIDC Payload Data in Returned Result

This configuration is used to enable the returning of the OIDC payload data to the API caller. If the OIDC RP Servlet is used, this setting should be set to "false". Enable this when the OIDC RP Servlet is not being used in the solution and the RP needs to use the returned OIDC tokens to call resources or manage the RP application session, etc. Refer to the *AccessMatrix REST API Specification* on the login response for more details on the data returned.

When this and the **Enable Auto Retrieval from Resource Endpoint** attribute are enabled, the resource info obtained is returned in the login response payload with `resource-info` as the key under the key-value pair of `responses` field.

> **Note**: This configuration is usually not needed when the OIDC RP Servlet is deployed as part of the solution.
>
> 1. In the **Advanced** tab, look for the **Include OIDC Payload Data in Returned Result** attribute.
> 2. Select "true" for the **Include OIDC Payload Data in Returned Result** attribute. Default is false.

Configure Connection Pool Settings

The OIDC Relying Party Lookup Module allows for the configuration of connection pool settings. Connection pooling refers to the method of creating a pool of connections and caching those connections so that it can be reused again.

> **Note:** For a list of all the parameters available, refer to *AccessMatrix Common Deployment Guide - AccessMatrix Core System Properties*.

To configure the connection pool parameters, perform the following steps:

1. Navigate to *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/* and open the **amsystem.properties** file.
2. Search for the `#== OIDC RP Lookup Module HC4Util Config===` comment.

3. Uncomment the parameters and specify the appropriate values as necessary. For example:

Properties (.properties files)

```
#== OIDC RP Lookup Module HC4Util Config===
#Uncomment connection pool settings
am.hc4.oidcrplookup.pool.maxconn_per_route=3
#am.hc4.oidcrplookup.pool.maxconn_total=
#am.hc4.oidcrplookup.pool.ttl_ms=
#Uncomment Retry settings
am.hc4.oidcrplookup.retry.times=4
#am.hc4.oidcrplookup.retry.request_sent_retry=
#Uncomment Timeout settings in ms
am.hc4.oidcrplookup.timeout.conn=40
#am.hc4.oidcrplookup.timeout.socket=
#am.hc4.oidcrplookup.timeout.conn_request=
#Uncomment Socket settings
#am.hc4.oidcrplookup.socket.receive-buffer-size=
#am.hc4.oidcrplookup.socket.send-buffer-size=
#am.hc4.oidcrplookup.socket.keep-alive=
#am.hc4.oidcrplookup.socket.linger=
am.hc4.oidcrplookup.socket.timeout=50
#am.hc4.oidcrplookup.socket.tcp-no-delay=
```

4. Save the changes.
5. Restart the AM5 server for the new changes to take effect.

Configure Protected Landing Page URL

The OIDC Relying Party Lookup Module supports the encryption of URLs for enhanced security and as a dynamic landing page. This ensures that URLs used in OIDC operations are protected from tampering or exposure.

An illustration of the flow for protecting URLs is as follows:

1. The URL that will be used as the dynamic landing page is encoded in Base64 format.
2. The am5/rest/authz/oauth/rp/url/protect endpoint is called to encrypt the Base64-encoded URL. For example:

> **Note:** For more details of this endpoint, refer to the *AccessMatrix REST API Specification*.

Request Example
JSON

Request Example

```
{
    "dynamicAgentSession": "true",
    "urlToBeProtected" : "aHR0cHM6Ly9nb29nbGUuY29t", //base-
64 encoded landing page
    "lookupModuleId" : "lookupModuleId"
}
```

Response Example (Success)

JSON

```
{
    "result": {
        "protectedUrl":
"http://localhost:8080/clp/rp/init/realm1?el=eyxVRUsxfUdtT1hzck1KWjZhM
mQ0OUhKeEpScjQwNnUrSERueXp0NkVDMUVuNWs4VTg9"
    },
    "amProcessingTimeMillis": 1
}
```

3. The returned protectedUrl parameter can be accessed via a web browser to start an RP-initiated OIDC flow.
4. The encrypted URL will redirect the user to the Common Login Page (CLP) for user authentication.
5. Upon successful authentication, the user will be directed to the actual URL.

Configure URL Protection Parameters

To configure the URL protection parameters, perform the following steps:

1. Navigate to *<ACMATRIX_ROOT>/tomcat/webapps/clp/WEB-INF/web.xml* and open the **web.xml** file.
2. Set the parameter UrlProtectionSupport to "true". For example:

XML

```
<context-param>
        <param-name>urlProtectionSupported</param-name>
        <param-value>true</param-value>  <!-- Default
is false -->
</context-param>
```

3. Save the changes.
4. Restart the AM5 server for the new changes to take effect.

Additional Configurations for OIDC RP Lookup Module

> **Note:** This assumes that you have already created an OIDC RP Lookup Module. Refer to the the instructions in Configure Lookup Module if you have not already done so.

1. Navigate to the **Administration Dashboard > Configuration > Authentication** and click on **User Lookup Module** in the left pane.
2. Click **Search** and select the relevant OIDC Lookup Module that will be configured.
3. Click on the **OIDC RP Servlet** tab.
4. Click the **Edit** button.
5. Specify the **Prefix for URL Protection** attribute with the format: *https://<hostname:port>/clp/rp/init/<realm_Id>*.

   > **Note:** Replace "<hostname:port>" with your publicly accessible AccessMatrix Server URI, and replace "<realmId>" with the ID of the authentication realm configured in the Configure Authentication Realm section.

6. Click the **Save** button.

5. OIDC Relying Party Implementation using API

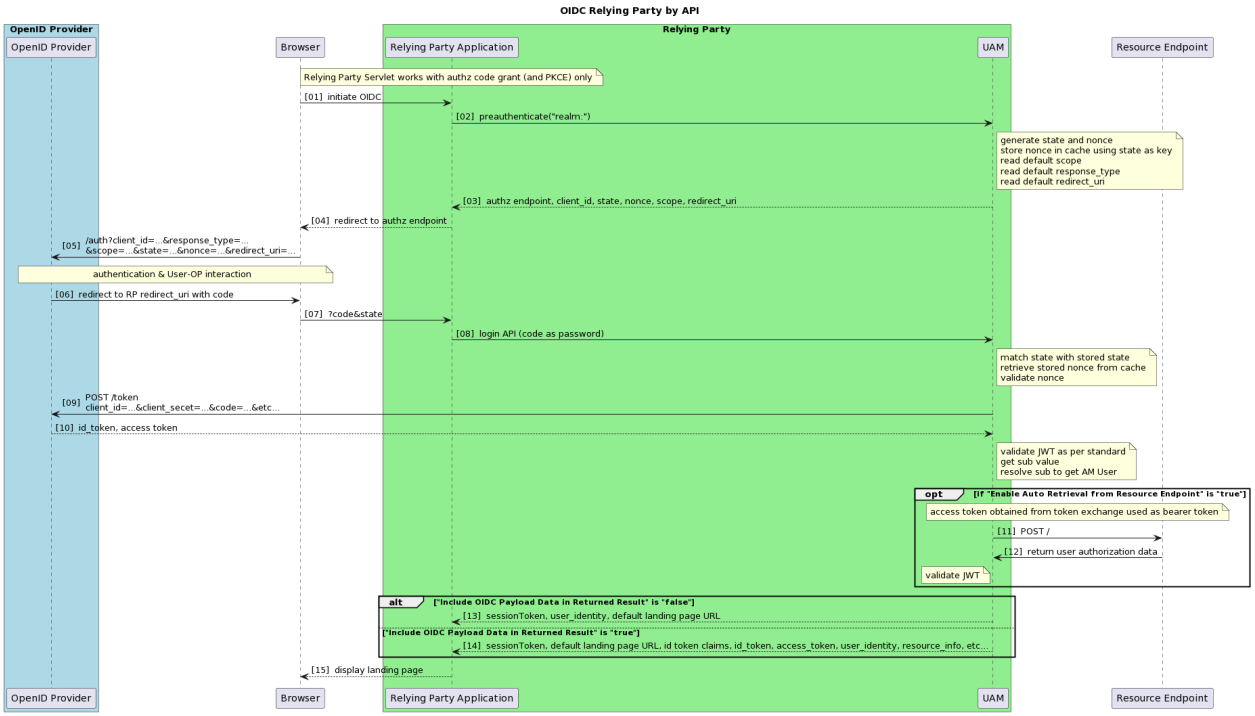
The following APIs are applicable for the Authentication use case only.

AccessMatrix as OIDC RP Backend for Authorization Code Flow

This section describes how UAM AccessMatrix can be used as an OIDC Relying Party for Authorization Code Flow. An Authorization Code Flow returns an authorization code to the Relying Party, which can then directly exchange it for an ID token and access token. This mechanism does not expose any tokens to the browser or end-user.

Sequence Diagram for OIDC Relying Party Lookup Module (For Authorization Code Flow)

- The Relying Party prepares an authentication request containing the desired request parameters.
- The Relying Party sends a request to the OpenID Provider to authenticate the end-user.
- The OpenID Provider authenticates the end-user and obtains authorization from the end-user to provide the requested scopes to the Relying Party.
- Once the end-user has been authenticated and has authorized the request, the OpenID Provider will return an authorization code to the Relying Party.
- The Relying Party contacts the token endpoint and exchanges the authorization code for an ID token identifying the end-user and optionally access and refresh tokens.



REST Sample Code

- Use /authn/preauthenticate to return the generated nonce and other information used to initiate authentication request to OpenID Provider.
- Use /authn/login for user login and to get access and ID token.
- Refer to the *AccessMatrix REST API Specification* for more details.

Preauthenticate Sample Request

Request Example

JSON

```
{
    "loginModuleId":"realm:oidc",
    "nonce":"CaVLk7t8UCV4gyU_" //optional
}
```

Response Example (Success)

JSON

```
{
    "result": {
        "authenticated":0,
```

Request Example


```
        "params":{
            "nonceStateExpiresAt":"1576567150473",

"authzEndpointPrepared":"http://192.168.50.53:8080/clp/auth?client_id=ZU1iWi
coF66HJ3nO2gXbng&response_type=code&scope=openid&redirect_uri=http%3A%2F%2F1
92.168.50.53%3A8081%2Ftest&nonce=CaVLk7t8UCV4gyU_&state=1RF2BRF7H3",
            "authzEndpoint":"http://192.168.50.53:8080/clp/auth",
            "nonce":"CaVLk7t8UCV4gyU_",
            "state":"1RF2BRF7H3"
        }
 },
    "amProcessingTimeMillis": 384
}
```


Possible Errors:

Exception Class	Error Code	Message	Possible Causes
AuthenticationException	10000	Failed to compute code challenge from code verifier	Failed to compute the code challenge from the code verifier (when PKCE flow is enabled).
		failed to encode param value of Authentication Request Query String for Authorization endpoint.	Failed to encode the param value of the Authentication Request Query String for the Authorization endpoint
AuthenticationException.OtherReasons	19999	AuthnRealm not found.	AuthnRealm not found.

Login Sample Request

Request Example

JSON


```
{
    "sessionToken":"",
    "userId":"oidc",
    "realmId":"oidc",

"password":"060001hAeH6rCMefR9KQsHL8ka5vr...truncated...RvaHezHrNhmwQjc8bA",
    "params":{
```

Request Example

```
        "state":"1RF2BRF7H3",
        "redirect_uri":"http://192.168.50.53:8081/test",
        "expectedNonce":"CaVLk7t8UCV4gyU_" //optional
    }
}
```

Response Example (Success)

JSON


```
{
    "result": {
        "attr.LT":"1576595431564",
        "authenticated":true,
        "sessionIID":"U0j631P-XQE9MboB1F7LnA",
        "lastAccessTime":1576566632052,
        "realmId":"oidc",
        "sessionFlags":1,
        "serverId":"demopc-D7050",
        "userId":"demouser",
        "attr.IT":"600",
        "expiresAt":1576595431,
        "realm.lastLoginSuccessTime":1576050674000,
        "loginTime":1576566631564,

"sessionToken":"10001PLOGJrqPns1U9...truncated...JYZAIPSy0OT93V4rX9dHfMtZ3",
        "responses":{
            "at_hash":"kQ8XkSDcLkXc3hUlVgAumg",
            "sub":"i2yNH-9vJA",

"id_token":"eyJraWQiOiJPQVMtTEx...truncated...ZLqfUFbimijMaxTcbLvg",
            "iss":"i-sprint.com",
            "nonce":"CaVLk7t8UCV4gyU_",
            "access_token":"060001c-_i4...truncated...TZOxN7C6jwVxGLQjA",
            "aud":"ZU1iWicoF66HJ3nO2gXbng",
            "acr":"1",
            "nbf":"1576566511",
            "auth_time":"1576566590",
            "user_identity":"i2yNH-9vJA",
            "exp":"1576570231",
            "iat":"1576566631",
            "jti":"j4CUe9t8DDM91JOIfrfGVw"
        },
        "idleTimeOutInSecs":600,
        "userUUID":"i2yNH-9vJA",
        "loginModuleStates":[
            {
                "authenticated":true,
                "canChangePassword":false,
                "id":"socialdu",
                "loginModuleHandlerClass":"SocialNetworkLogin"
            }
        ],
        "attr.regenerated":"1576566632053"
```

Request Example

```
    },
    "amProcessingTimeMillis": 493
}
```

Possible Errors:

Exception Class	Error Code	Message	Possible Causes
AuthenticationException.VerificationFailed	10020	Failed to verify state	The state has expired or does not exist.
		< OP returned response's error_description field>	The OP returned the invalid_grant error.
		Half of hash of access_token does not match at_hash, algo=<at_has hash algorithm>	Failed to validate the access token hash with the ID token at_hash claim.
AuthenticationException.OtherReasons	19999	Authorization code not provided	Authorization code was not provided.
		code_verifier is not found in cache.	The code_verifier was not found in the cache (when PKCE flow is enabled).
		state param not provided.	The state param was not provided (when PKCE flow is enabled).
		redirect_uri param must be specified if authorization code is used	The redirect_uri param was not specified (when authorization code is used).
		Failed to prepare client authn to token endpoint	Failed to prepare client authentication to Token endpoint.

Exception Class	Error Code	Message	Possible Causes
		Failed to call OIDC Provider server	Failed to call the OIDC OpenID Provider server (e.g. Token endpoint).
		response content is not JSON	The OP response content is not in the JSON format (when calling Token endpoint).
		< OP returned response's error field>,< OP returned response's error_description field>	An error was returned by the OP. Refer to the actual error and description provided in the message for details.
		Unexpected response content	Some other unexpected error was encountered when parsing the OP Token endpoint response content.
		Failed to parse token exchange response	Failed to parse the OP token exchange response when the returned status code is 200.
		Failed to decrypt JWE ID Token	Failed to decrypt an encrypted JWE ID token while verifying it.
		Invalid JWT	Invalid JWT encountered when verifying the ID token signature.
		Failed to get signature information	Failed to get signature information from the ID token for at_hash validation.

Exception Class	Error Code	Message	Possible Causes
		Unexpected hash algo for access token hash calculation	The at_hash hash algorithm is null.
		Invalid issuer: <IdToken's Issuer>, expects: " <module OIDC metadata 's Issuer>	Invalid ID token issuer encountered during validation of ID token.
		Failed to get issuer information	Failed to get issuer information from ID token.
		JWT idTokenClaims are not complete or have wrong type	Failed to get claim for identity mapping from ID token.
		ID Token Claim Name specified for user mapping not found in ID Token	The ID token claim name specified for user mapping was not found in the ID token.
		claim not found in id token to resolve external id	The claim used to resolve the external ID was not found in the ID token (when plugin is not used to resolve user identity)
		Segment not found for segmentId=<params.segmentId>, storeId= <module user store id>	Failed to resolve segment ID passed in from API params.
		<number>users found with storeId=<module user store id>, external Id=<resolved user identity>	More than one mapping was found (when identity mapping is configured for user identity mapping).
		Fail to look up user from Identity Mapping for External Id=<resolved user	Failed to retrieve mapping from identity mapping table (when identity mapping is

Exception Class	Error Code	Message	Possible Causes
		identity>, error=<error message>	configured for user identity mapping).
		Auto provisioning has failed as 'User Mapping Option' is set to '<userIdentityMapping attribute value> '. The User Mapping Lookup Option must be set to 'Identity Mapping' if auto provisioning is enabled	Auto provisioning has failed as the 'User Mapping Option' is set to '<userIdentityMapping attribute value> '. The 'User Mapping Lookup Option' must be set to "Identity Mapping" if auto provisioning is enabled.
		Failed to call Resource Endpoint URL	Failed to call Resource endpoint URL.
		Resource endpoint response status code is NOT 200, the error reference id is "<AM5's errorRefId>". Please contact administrator for further assistance.	Resource endpoint response status code is NOT 200. The error reference Id is "<errorRefId>".
		Invalid resource JWT	Invalid resource JWT due to failed signature verification or other verification errors.
		Failed to parse resource response	Failed to parse resource response due to expected error.
		Expected issuer of resource JWT is not matched.	The expected issuer of resource JWT is not matched.
		Expected audience of resource JWT is not matched.	The expected audience of resource JWT is not matched.
		resource JWT is already expired.	The resource JWT has already expired.

Exception Class	Error Code	Message	Possible Causes
		Invalid claim. Failed to verify resource JWT Invalid claim.	Failed to verify resource JWT.
		resource JWT Claims obtained is null	The resource JWT claims obtained from the Resource endpoint are null.
AuthenticationException.UserNotFound	10001	User not found, user_identity=<user identity>	AM cannot find the user and the auto create user feature is not enabled in the lookup module.
AuthenticationException.AutoProvisioningFailed	10107	Fail to create User for External Id App=<lookup module Id", External Id=<resolved user identity>	The UserUUID returned is null after calling the auto provisioning plugin.
AuthenticationException	10000	Failed authorization check with resource info	Failed authorization check with resource info when the IResolveUserIdentity Plugin's checkAuthzWithResourceInfo is not successful but authenticationException is null.

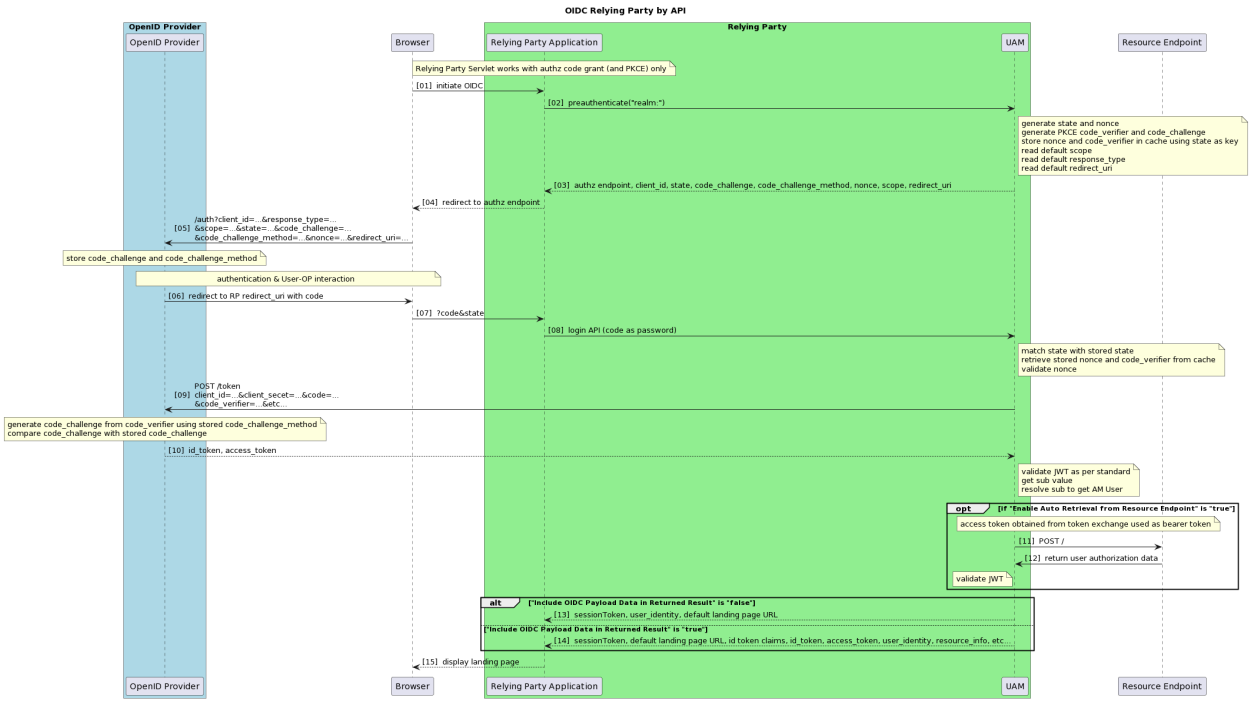
AccessMatrix as OIDC RP Backend for Authorization Code Flow with PKCE

This section describes how UAM AccessMatrix can be used as the OIDC Relying Party for the Authorization Code flow with PKCE. In this flow, the OAuth Client generates a cryptographically random key called a code verifier and derives from it a transformed value called a code challenge. This code challenge, along with the transformation method, is sent to the OpenID Provider in exchange for an authorization code. This code can then be sent to the OP's token endpoint along with the original code verifier in exchange for an ID token and access token.

This flow does not expose any tokens to the browser or end-user and is more resilient to CSFR and authorization code interception attacks. Thus, it is highly recommended to use this flow instead of the Authorization Code flow when possible.

Sequence Diagram for OIDC Relying Party Lookup Module (For Authorization Code Flow with PKCE)

- The Relying Party prepares an authentication request containing the desired request parameters, including state, code_challenge and code_challenge_method.
- The Relying Party sends a request to the OpenID Provider to authenticate the end-user.
- The OpenID Provider authenticates the end-user and obtains authorization from the end-user to provide the requested scopes to the Relying Party.
- Once the end-user has been authenticated and has authorized the request, the OpenID Provider will return an authorization code to the Relying Party.
- The Relying Party contacts the token endpoint and exchanges the authorization code, along with the the original code verifier, for an ID token identifying the end-user. Optionally, access and refresh tokens are returned as well.



REST Sample Code

- Use /authn/preauthenticate to return the generated nonce and other information used to initiate authentication request to OpenID Provider.
- Use /authn/login for user login and to get access and ID token.
- Refer to the *AccessMatrix REST API Specification* for more details.

Preauthenticate Sample Request

Request Example

JSON

```
{
    "loginModuleId":"realm:oidc",
    "params" :{
    "nonce":"CaVLk7t8UCV4gyU_" //optional
    }
}
```

Response Example (Success)

JSON

```
{
    "result": {
        "authenticated":0,
        "params":{
            "nonceStateExpiresAt":"1576567150473",

"authzEndpointPrepared":"http://192.168.50.53:8080/clp/auth?client_id=ZU1iWicoF66HJ3nO2gXbng&response_type=code&scope=openid&redirect_uri=http%3A%2F%2F192.168.50.53%3A8081%2Ftest&nonce=CaVLk7t8UCV4gyU_&state=1RF2BRF7H3&code_challenge=HtKdpT%2B9ebYA%2Fai5uPugtKv%2FIX%2FUyn1y5cUDnWsyEiQ%3D&code_challenge_method=S256",
            "authzEndpoint":"http://192.168.50.53:8080/clp/auth",
            "nonce":"CaVLk7t8UCV4gyU_",
            "state":"1RF2BRF7H3"
        }
 },
    "amProcessingTimeMillis": 384
}
```

Possible Errors:

Same as Authorization Code flow.

Login Sample Request

Request Example

JSON


```
{
    "sessionToken":"",
    "userId":"oidc",
    "realmId":"oidc",

"password":"060001hAeH6rCMefR9KQsHL8ka5vr...truncated...RvaHezHrNhmwQjc8bA",
    "params":{
        "state":"1RF2BRF7H3",
        "redirect_uri":"http://192.168.50.53:8081/test",
        "expectedNonce":"CaVLk7t8UCV4gyU_" //optional
    }
}
```


Response Example (Success)

JSON


```
{
    "result": {
        "attr.LT":"1576595431564",
        "authenticated":true,
        "sessionIID":"U0j631P-XQE9MboB1F7LnA",
        "lastAccessTime":1576566632052,
        "realmId":"oidc",
        "sessionFlags":1,
        "serverId":"demopc-D7050",
        "userId":"demouser",
        "attr.IT":"600",
        "expiresAt":1576595431,
        "realm.lastLoginSuccessTime":1576050674000,
        "loginTime":1576566631564,

"sessionToken":"10001PLOGJrqPns1U9...truncated...JYZAIPSy0OT93V4rX9dHfMtZ3",
        "responses":{
            "at_hash":"kQ8XkSDcLkXc3hUlVgAumg",
            "sub":"i2yNH-9vJA",

"id_token":"eyJraWQiOiJPQVMtTEx...truncated...ZLqfUFbimijMaxTcbLvg",
            "iss":"i-sprint.com",
            "nonce":"CaVLk7t8UCV4gyU_",
            "access_token":"060001c-_i4...truncated...TZOxN7C6jwVxGLQjA",
            "aud":"ZU1iWicoF66HJ3nO2gXbng",
            "acr":"1",
            "nbf":"1576566511",
            "auth_time":"1576566590",
            "user_identity":"i2yNH-9vJA",
            "exp":"1576570231",
            "iat":"1576566631",
            "jti":"j4CUe9t8DDM91JOIfrfGVw"
```

Request Example


```
        },
        "idleTimeOutInSecs":600,
        "userUUID":"i2yNH-9vJA",
        "loginModuleStates":[
            {
                "authenticated":true,
                "canChangePassword":false,
                "id":"socialdu",
                "loginModuleHandlerClass":"SocialNetworkLogin"
            }
        ],
        "attr.regenerated":"1576566632053"
    },
    "amProcessingTimeMillis": 493
}
```


Possible Errors:

Same as Authorization Code flow.

6. UAM OIDC Relying Party Reports

Federated Session Events Audit Report

The AccessMatrix UAM authorization server audits important OpenID Connect (OIDC) Relying Party federated session events, such as the login, logout, and session refresh events. Thereafter, the administrator can create a report to view the audited events.

> **Note**: The following audit events require session management features to be enabled. Refer to Common Configurations - Configure Session Management for instructions on how to do so.

Currently, AccessMatrix UAM OIDC supports the following audit events:

Event Type	Description	Fields
FederatedSession-Add	This audit event is raised after the user initiates the login process via the OIDC Relying Party application, and the OIDC token(s) received from the OpenID Provider are subsequently cached in the UAM server.	sessionIId
		realmId
		tokenType
		token
		accessTokenExpiryTime
		tokenExpiryTime
FederatedSession-Delete	This audit event is raised after the user logs out from the OIDC Relying Party application, and any persisted OIDC token(s) associated with the user's federated session(s) are removed from the UAM server.	sessionIId
		realmId
FederatedSession-Refresh	This audit event is raised after the OIDC token(s) associated with the user's federated session are refreshed.	sessionIId
		realmId
		tokenType
		token
		accessTokenExpiryTime

Event Type	Description	Fields
		tokenExpiryTime

Note: Refer to *AccessMatrix Common Administration Guide - Audit Policy* for more information.

The above events can be viewed by generating either an Events Report or User Activities Report in the Admin Console:

1. Navigate to either:
    a. Events Report - **Report Dashboard > System & Admin > Historical > Events**.
    b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities**.
2. For **Include actions done by background tasks**, select "Yes" from the drop-down list.
3. For **Event Category**, ensure **All Event Categories** is selected.
    a. Alternatively, click **Select Event Category** and select **FederatedSession**.
4. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

Expired RP Token Housekeeping Report

If session management is enabled for the OIDC RP lookup module, users' OIDC RP tokens (such as access and expiry tokens) are regularly persisted. By default, expired persisted RP tokens are removed when the RP application calls the AM5 Logout API to terminate user sessions.

However, the AccessMatrix server also routinely removes the expired tokens at a fixed time each day (3 AM, server time) via a backend scheduler. These events are audited under the following categories:

Event Type	Description	Fields
ExpiredRPTokenCleaner-Initialized	This audit event is raised when the Expired RP Token Cleaner is initialized.	nextExecTime
ExpiredRPTokenCleaner-Started	This audit event is raised when the Expired RP Token Cleaner begins purging expired RP tokens.	startTime
		nextExecTime

Event Type	Description	Fields
ExpiredRPTokenCleaner-Completed	This audit event is raised when the Expired RP Token Cleaner has completed the purging of expired RP tokens.	expiredRPTokenDetail
		startTime
		endTime
		elapsedTime
		nextExecTime

The above events can be viewed by generating either an Events Report or User Activities Report in the Admin Console:

1. Navigate to either:
    a. Events Report - **Report Dashboard > System & Admin > Historical > Events**.
    b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities**.
2. For **Include actions done by background tasks**, select "Yes" from the drop-down list.
3. For **Event Category**, ensure **All Event Categories** is selected.
    a. Alternatively, click **Select Event Category** and select **ExpiredRPTokenCleaner**.
4. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

7. Appendices

A. Microsoft Entra ID as OIDC OP

The following sections provide additional notes and information on using Microsoft Entra ID as the OIDC OP:

A.1 Enable Password-less Phone Sign-in for a Registered OIDC App

After registering your app in the Microsoft Entra ID app registrations portal, you can create conditional access policies to restrict access to the app to the specified authentication methods.

The following steps provide instructions on how to do so, using password-less phone sign-in as the authentication method as an example:

Create a custom authentication strength:

1. In the Microsoft Azure portal web portal, navigate to **Microsoft Entra ID > Security > Authentication methods > Authentication strengths (Preview) > New authentication strength**.
2. Provide an appropriate **Name** and **Description**.
3. Select any of the available methods you want to allow, e.g. **Microsoft Authenticator (Phone Sign-in)**.

   > **Note**: Refer to https://learn.microsoft.com/en-us/azure/active-directory/conditional-access/concept-conditional-access-grant for more details on this.

4. Click **Next** and review the policy configurations to complete the creation process.

Create new conditional access policy:

1. Navigate to **Microsoft Entra ID > Security > Conditional Access > New policy > Create new policy**.
2. Under **Cloud apps or actions**, select your registered app.
3. Under **Grant**, select the **Grant access** radio button and check the **Require authentication strength (Preview)** checkbox.
4. Select the custom authentication strength you created.

Enable Microsoft Authenticator for users:

1. Navigate to **Microsoft Entra ID > Security > Authentication methods > Policies** and click on Microsoft Authenticator.
2. Under **ENABLE**, choose **Yes**.
3. Under **TARGET**, choose either **All users** or **Select users** as appropriate.
4. Under **Authentication mode**, for all appropriate users, choose "Any" or "Passwordless" depending on your requirements.

A.2 User Authentication Resources

The following resources provide useful information for end users on how to install and enable the use of Microsoft Authenticator for passwordless phone sign-ins:

1. Register Microsoft Authenticator with seeds provisioned by Microsoft:
    a. https://learn.microsoft.com/en-us/azure/active-directory/authentication/concept-registration-mfa-sspr-combined
    b. https://learn.microsoft.com/en-us/azure/active-directory/authentication/howto-registration-mfa-sspr-combined
2. Enable passwordless phone sign-in:
    a. https://learn.microsoft.com/en-us/azure/active-directory/authentication/howto-authentication-passwordless-phone

A.3 Errors & Troubleshooting

If a user has lost their phone or uninstalled Microsoft Authenticator:

1. In the Microsoft Azure web portal, navigate to **Microsoft Entra ID > Users > All Users**.
2. Select the relevant user and click **Authentication Methods**.
3. Click **Require re-register MFA**.

If a user experiences log-in errors due to changes in authentication methods:

1. In the Microsoft Azure web portal, navigate to **Microsoft Entra ID > Users > All Users**.
2. Select the relevant user and click **Authentication Methods**.

3. Click **Revoke MFA sessions**.

If a user is required to do phone sign-in but has not completed the necessary set-up steps:

1. Assuming the steps in the previous section have been completed by the administrator, the user should automatically be prompted to set up their phone passwordless sign-in method after attempting to log in to the app. Once the setup is complete, the user can use the phone sign-in method to authenticate to the OIDC app.

i	**Note**: Authenticator push cannot be used with phone sign-in, nor can they be switched from one to the other interchangeably. If the authentication method is switched to the authenticator push method, the login page does not successfully redirect to the appropriate landing page (although the authentication method may still function). After switching back to phone sign-in, the user will continue to encounter errors. In this scenario, the administrator must revoke their MFA sessions as described in the steps above.